

Applied Artificial Intelligence and Deep Learning Book

Tim Mayer

Biplov Bhandari

David Saah

2025-09-18

Table of contents

Introduction

The NASA EarthRISE program harnesses NASA Earth Action capabilities to deliver trusted solutions for sustained benefits to society. Collaborating with State and Local partners and striving to strengthen the capacities of our partners to use satellite data and geospatial technology to address critical challenges. EarthRISE co-develops innovative solutions through a network of partners to improve resilience and sustainable resource management across scales. Additionally, EarthRISE focuses on developing participants in innovative knowledge products such as the SAR Handbook and the GEE book designed to support capacity building in applying Remote Sensing and geospatial approaches to address challenges.

The focus of the EarthRISE Applied Artificial Intelligence and Deep Learning Book is to provide practitioners with a wide variety of applied examples of Remote Sensing Artificial Intelligence and Deep Learning approaches. With each chapter focusing on a specific problem set such as object detection or downscaling using Deep Learning. Additionally, throughout the book's chapters various examples are provided spanning the aforementioned NASA Earth Action thematic areas. Thereby providing a wide variety of thematic applications to complement reader's domain specific practical knowledge such as agronomy or forestry etc.

We suspect readers are coming to this virtual book with preexisting geospatial expertise. However, limited Artificial Intelligence and Deep Learning knowledge

Each chapter contains both the theoretical background as well as a practical hand-on section facilitated through virtual notebooks. Finally, this book spans a variety of platforms such as TensorFlow and PyTorch to provide readers with a wide set of examples.

Part I

Semantic Segmentation

1 Semantic Segmentation

1.1 Rice mapping in Bhutan with U-Net using high resolution satellite imagery



Run in Colab



View on GitHub

1.2 Note: This notebook is meant to be run in Colab. You can still run this locally, make sure you have [Google Drive API](#) installed and path adjusted in relevant places.

This notebook is also available in this github repo: <https://github.com/SERVIR/servir-aces>. Navigate to the notebook folder.

1.3 Setup environment

```
from google.colab import drive
drive.mount("/content/drive")
```

```
!pip install servir-aces
```

Collecting servir-aces

Downloading servir_aces-0.0.14-py2.py3-none-any.whl (32 kB)

Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from servir-aces)

Requirement already satisfied: tensorflow>=2.9.3 in /usr/local/lib/python3.10/dist-packages (from servir-aces)

Requirement already satisfied: earthengine-api in /usr/local/lib/python3.10/dist-packages (from servir-aces)

Collecting python-dotenv>=1.0.0 (from servir-aces)

Downloading python_dotenv-1.0.1-py3-none-any.whl (19 kB)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from servir-aces)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: flatbuffers>=23.5.26 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: ml-dtypes~=0.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: wrapt<1.15,>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: tensorboard<2.16,>=2.15 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: tensorflow-estimator<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: keras<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-cloud-storage in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-api-python-client>=1.12.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-auth>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-auth-http2>=0.0.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: http2<1dev,>=0.9.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.0,<3.0.0dev,>=1.30.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: google-auth-oauthlib<2,>=0.5 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.10.0)

```

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: google-cloud-core<3.0dev,>=2.3.0 in /usr/local/lib/python3.10,
Requirement already satisfied: google-resumable-media>=2.3.2 in /usr/local/lib/python3.10/di
Requirement already satisfied: googleapis-common-protos<2.0.dev0,>=1.56.2 in /usr/local/lib/p
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: google-crc32c<2.0dev,>=1.0 in /usr/local/lib/python3.10/dist-j
Requirement already satisfied: pyasn1<0.7.0,>=0.4.6 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (f
Installing collected packages: python-dotenv, servir-aces
Successfully installed python-dotenv-1.0.1 servir-aces-0.0.14

```

```
!git clone https://github.com/SERVIR/servir-aces
```

```

Cloning into 'servir-aces'...
remote: Enumerating objects: 740, done.
remote: Counting objects: 100% (116/116), done.
remote: Compressing objects: 100% (78/78), done.
remote: Total 740 (delta 46), reused 68 (delta 38), pack-reused 624
Receiving objects: 100% (740/740), 5.07 MiB | 16.12 MiB/s, done.
Resolving deltas: 100% (468/468), done.

```

1.3.1 Download datasets

For this chapter, we have already prepared and exported the training datasets. They can be found at the google cloud storage and we will use `gsutil` to get the dataset in our workspace. The dataset has `training`, `testing`, and `validation` subdirectory. Let's start by downloading these datasets in our workspace.

If you're looking to produce your own datasets, you can follow this [notebook](#) which was used to produce these training, testing, and validation datasets provided in this notebook.

```
!mkdir -p content/datasets
```

```
!gsutil -m cp -r gs://dl-book/chapter-1 content/datasets/
```

If you experience problems with multiprocessing on MacOS, they might be related to <https://b>

```
Copying gs://dl-book/chapter-1/.DS_Store...
Copying gs://dl-book/chapter-1/dnn_planet_wo_indices/testing/testing.tfrecord.gz...
Copying gs://dl-book/chapter-1/dnn_planet_wo_indices/training/training.tfrecord.gz...
Copying gs://dl-book/chapter-1/dnn_planet_wo_indices/validation/validation.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_202100000.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_202100001.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_202100002.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_202100003.tfrecord.gz...
Copying gs://dl-book/chapter-1/models/dnn_v1/config.env...
Copying gs://dl-book/chapter-1/images/image_202100004.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_202100005.tfrecord.gz...
Copying gs://dl-book/chapter-1/images/image_2021mixer.json...
Copying gs://dl-book/chapter-1/models/dnn_v1/aces/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/aces/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/aces/saved_model.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/aces/variables/variables.data-00000-of-00001...
Copying gs://dl-book/chapter-1/models/dnn_v1/aces/variables/variables.index...
Copying gs://dl-book/chapter-1/models/dnn_v1/config.json...
Copying gs://dl-book/chapter-1/models/dnn_v1/evaluation.txt...
Copying gs://dl-book/chapter-1/models/dnn_v1/logs/train/events.out.tfevents.1713307528.b5c4c...
Copying gs://dl-book/chapter-1/models/dnn_v1/logs/validation/events.out.tfevents.1713307537.l...
Copying gs://dl-book/chapter-1/models/dnn_v1/model.png...
Copying gs://dl-book/chapter-1/models/dnn_v1/model.txt...
Copying gs://dl-book/chapter-1/models/dnn_v1/modelCheckpoint/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/modelCheckpoint/variables/variables.data-00000-...
Copying gs://dl-book/chapter-1/models/dnn_v1/modelCheckpoint/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/aces/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/aces/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/model.pkl...
Copying gs://dl-book/chapter-1/models/dnn_v1/modelCheckpoint/variables/variables.index...
Copying gs://dl-book/chapter-1/models/dnn_v1/parameters.txt...
Copying gs://dl-book/chapter-1/models/dnn_v1/prediction/prediction_dnn_v1.TFRecord...
Copying gs://dl-book/chapter-1/models/unet_v1/model.pkl...
Copying gs://dl-book/chapter-1/models/dnn_v1/trained-model/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/modelCheckpoint/saved_model.pb...
==> NOTE: You are downloading one or more large file(s), which would
run significantly faster if you enabled sliced object downloads. This
feature is enabled by default but requires that compiled crcmod be
installed (see "gsutil help crcmod").
```

```
Copying gs://dl-book/chapter-1/models/unet_v1/aces/saved_model.pb...
```


Copying gs://dl-book/chapter-1/models/dnn_v1/trained-model/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/trained-model/saved_model.pb...
Copying gs://dl-book/chapter-1/models/dnn_v1/trained-model/variables/variables.index...
Copying gs://dl-book/chapter-1/models/dnn_v1/trained-model/variables/variables.data-00000-of-...
Copying gs://dl-book/chapter-1/models/dnn_v1/training.png...
Copying gs://dl-book/chapter-1/models/unet_v1/aces/variables/variables.data-00000-of-00001...
Copying gs://dl-book/chapter-1/models/unet_v1/model.png...
Copying gs://dl-book/chapter-1/models/unet_v1/logs/train/events.out.tfevents.1713299324.b5c4...
Copying gs://dl-book/chapter-1/models/unet_v1/config.env...
Copying gs://dl-book/chapter-1/models/unet_v1/evaluation.txt...
Copying gs://dl-book/chapter-1/models/unet_v1/config.json...
Copying gs://dl-book/chapter-1/models/unet_v1/model.txt...
Copying gs://dl-book/chapter-1/models/unet_v1/logs/validation/events.out.tfevents.1713299558...
Copying gs://dl-book/chapter-1/models/unet_v1/aces/variables/variables.index...
Copying gs://dl-book/chapter-1/models/unet_v1/modelCheckpoint/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/modelCheckpoint/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/modelCheckpoint/saved_model.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/modelCheckpoint/variables/variables.data-00000-of-...
Copying gs://dl-book/chapter-1/models/unet_v1/modelCheckpoint/variables/variables.index...
Copying gs://dl-book/chapter-1/models/unet_v1/prediction/prediction_unet_v1.TFRecord...
Copying gs://dl-book/chapter-1/models/unet_v1/parameters.txt...
Copying gs://dl-book/chapter-1/models/unet_v1/trained-model/fingerprint.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/trained-model/keras_metadata.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/trained-model/saved_model.pb...
Copying gs://dl-book/chapter-1/models/unet_v1/trained-model/variables/variables.data-00000-of-...
Copying gs://dl-book/chapter-1/models/unet_v1/trained-model/variables/variables.index...
Copying gs://dl-book/chapter-1/models/unet_v1/training.png...
Copying gs://dl-book/chapter-1/prediction/prediction_dnn_v1.TFRecord...
Copying gs://dl-book/chapter-1/prediction/prediction_unet_v1.TFRecord...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00000-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00001-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00003-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00002-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00004-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00005-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00006-of-00008.tfrec...
Copying gs://dl-book/chapter-1/training_data/testing_10/testing__256x256-00007-of-00008.tfrec...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00000-of-00038...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00001-of-00038...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00002-of-00038...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00003-of-00038...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00004-of-00038...
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/testing/testing-00005-of-00038...

[illegible]

[illegible]

```

Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00016-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00017-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00018-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00019-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00020-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00021-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00022-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00023-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00024-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00025-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00026-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00027-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00028-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00029-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00030-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00031-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00032-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00033-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00034-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00035-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00036-of-
Copying gs://dl-book/chapter-1/unet_256x256_planet_wo_indices/validation/validation-00037-of-
/ [187/192 files][ 16.3 GiB/ 16.3 GiB] 99% Done 43.1 MiB/s ETA 00:00:00

```

1.3.2 Setup config file variables

Now the repo is downloaded. We will create an environment file file to place point to our training data and customize parameters for the model. To do this, we make a copy of the `.env.example` file provided.

Under the hood, all the configuration provided via the environment file are parsed as a config object and can be accessed programatically.

Note current version does not expose all the model intracacies through the environment file but future version may include those depending on the need.

```
!cp servir-aces/.env.example servir-aces/config.env
```

Okay, now we have the `config.env` file, we will use this to provide our environments and parameters.

Note there are several parameters that can be changed. Let's start by changing the `BASEDIR` and `OUTPUT_DIR` as below.

```
BASEDIR = "/content/"
OUTPUT_DIR = "/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output"
```

We will start by training a [U-Net](#) model using the `dl-book/chapter-1/unet_256x256_planet_wo_indices` dataset inside the `dataset` folder for this exercise. Let's go ahead and change our `DATADIR` in the `config.env` file as below.

```
DATADIR = "datasets/unet_256x256_planet_wo_indices"
```

These datasets have RGBN from PlanetScope mosaic. Since we are trying to map the rice fields, we use growing season and pre-growing season information. Thus, we have 8 optical bands, namely `red_before`, `green_before`, `blue_before`, `nir_before`, `red_during`, `green_during`, `blue_during`, and `nir_during`. In addition, you can use `USE_ELEVATION` and `USE_S1` config to include the topographic and radar information. Since this datasets have topographic and radar features, so we won't be setting these config values. Similarly, these datasets are tiled to 256x256 pixels, so let's also change that.

```
# For model training, USE_ELEVATION extends FEATURES with "elevation" & "slope"
# USE_S1 extends FEATURES with "vv_asc_before", "vh_asc_before", "vv_asc_during", "vh_asc_during",
# "vv_desc_before", "vh_desc_before", "vv_desc_during", "vh_desc_during"
# In case these are not useful and you have other bands in your training data, you can do set
# USE_ELEVATION and USE_S1 to False and update FEATURES to include needed bands
USE_ELEVATION = False
USE_S1 = False

PATCH_SHAPE = (256, 256)
```

Next, we need to calculate the size of the training, testing and validation dataset. For this, we know our size before hand. But `aces` also provides handful of functions that we can use to calculate this. See this [notebook](#) to learn more about how to do it. We will also change the `BATCH_SIZE` to 32; if you have larger memory available, you can increase the `BATCH_SIZE`. You can run for longer `EPOCHS` by changing the `EPOCHS` paramter; we will keep it to 5 for now.

```
# Sizes of the training and evaluation datasets.
TRAIN_SIZE = 8531
TEST_SIZE = 1222
VAL_SIZE = 2404
BATCH_SIZE = 32
EPOCHS = 30
```

1.3.3 Update the config file programtically

We can also make a dictionary so we can change these config settings programatically.

```
BASEDIR    = "/content/" # @param {type:"string"}
OUTPUT_DIR = "/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output" # @param {type:"string"}
DATADIR    = "datasets/unet_256x256_planet_wo_indices" # @param {type:"string"}
# PATCH_SHAPE, USE_ELEVATION, USE_S1, TRAIN_SIZE, TEST_SIZE, VAL_SIZE
# BATCH_SIZE, EPOCHS are converted to their appropriate type.
USE_ELEVATION = "False" # @param {type:"string"}
USE_S1        = "False" # @param {type:"string"}
PATCH_SHAPE  = "(256, 256)" # @param {type:"string"}
TRAIN_SIZE    = "8531" # @param {type:"string"}
TEST_SIZE     = "1222" # @param {type:"string"}
VAL_SIZE      = "2404" # @param {type:"string"}
BATCH_SIZE    = "32" # @param {type:"string"}
EPOCHS        = "30" # @param {type:"string"}
MODEL_DIR_NAME = "unet_v1" # @param {type:"string"}
```

```
unet_config_settings = {
    "BASEDIR" : BASEDIR,
    "OUTPUT_DIR": OUTPUT_DIR,
    "DATADIR": DATADIR,
    "USE_ELEVATION": USE_ELEVATION,
    "USE_S1": USE_S1,
    "PATCH_SHAPE": PATCH_SHAPE,
    "TRAIN_SIZE": TRAIN_SIZE,
    "TEST_SIZE": TEST_SIZE,
    "VAL_SIZE": VAL_SIZE,
    "BATCH_SIZE": BATCH_SIZE,
    "EPOCHS": EPOCHS,
    "MODEL_DIR_NAME": MODEL_DIR_NAME,
}
```

```
import dotenv

config_file = "servir-aces/config.env"

for config_key in unet_config_settings:
    dotenv.set_key(dotenv_path=config_file,
                   key_to_set=config_key,
                   value_to_set=unet_config_settings[config_key]
                   )
```

1.4 U-Net Model

1.4.1 Load config file variables

```
from aces import Config, DataProcessor, ModelTrainer, EEUtils
```

Let's load our config file through the `Config` class.

```
unet_config = Config(config_file=config_file)
```

```
BASEDIR: /content
```

```
DATADIR: /content/datasets/unet_256x256_planet_wo_indices
```

```
using features: ['red_before', 'green_before', 'blue_before', 'nir_before', 'red_during', 'g
```

```
using labels: ['class']
```

Most of the config in the `config.env` is now available via the config instance. Let's check few of them here.

```
unet_config.TRAINING_DIR, unet_config.OUTPUT_DIR, unet_config.BATCH_SIZE, unet_config.TRAIN_S
```

```
(PosixPath('/content/datasets/unet_256x256_planet_wo_indices/training'),  
 PosixPath('/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output'),  
 32,  
 8531)
```

1.4.2 Load ModelTrainer class

Next, let's make an instance of the `ModelTrainer` object. The `ModelTrainer` class provides various tools for training, buidling, compiling, and running specified deep learning models.

```
unet_model_trainer = ModelTrainer(unet_config, seed=42)
```

Using seed: 42

1.4.3 Train and Save U-Net model

`ModelTrainer` class provides various functionality. We will use `train_model` function that helps to train the model using the provided configuration settings.

This method performs the following steps: - Configures memory growth for TensorFlow. - Creates TensorFlow datasets for training, testing, and validation. - Builds and compiles the model. - Prepares the output directory for saving models and results. - Starts the training process. - Evaluates and prints validation metrics. - Saves training parameters, plots, and models.

```
unet_model_trainer.train_model()
```

```
*****
***** Clear Session... *****
*****
***** Configure memory growth... *****
> Found 1 GPUs
*****
***** creating datasets... *****
Loading dataset from /content/datasets/unet_256x256_planet_wo_indices/training/*
randomly transforming data
Loading dataset from /content/datasets/unet_256x256_planet_wo_indices/validation/*
Loading dataset from /content/datasets/unet_256x256_planet_wo_indices/testing/*
Printing dataset info:
Training
inputs: float32 (32, 256, 256, 8)
tf.Tensor(
[[[0.073075 0.063275 0.0411    ... 0.050625 0.0274    0.23925 ]
  [0.084775 0.067375 0.047025 ... 0.057675 0.032075 0.242375]
  [0.083625 0.068575 0.045075 ... 0.059275 0.0332    0.2409  ]
  ...
  [0.0702    0.06825  0.04495  ... 0.055025 0.028325 0.26305 ]
  [0.064475 0.066     0.043575 ... 0.0524    0.027075 0.26705 ]
  [0.0676    0.06355  0.04535  ... 0.05375  0.02875  0.263275]]

[[0.071475 0.062225 0.0388    ... 0.0496    0.025375 0.24155 ]
 [0.07815  0.065025 0.044225 ... 0.0545    0.02905  0.24175 ]
 [0.086025 0.069125 0.046175 ... 0.05855   0.0326    0.2355  ]
  ...
 [0.060775 0.0627    0.041875 ... 0.051575 0.029725 0.267475]
 [0.061375 0.06225  0.04225  ... 0.0513    0.02685  0.268375]
 [0.06845  0.064075 0.043925 ... 0.052925 0.028575 0.267975]]
```



```

[[0.0677  0.0605  0.038625 ... 0.04835  0.024825 0.236075]
 [0.078375 0.0629  0.04215  ... 0.0524   0.02855  0.237375]
 [0.0857   0.065725 0.04635  ... 0.05705  0.030975 0.235375]
 ...
 [0.07      0.062775 0.04485  ... 0.053425 0.0292   0.27015 ]
 [0.0607   0.060675 0.041175 ... 0.053075 0.026275 0.27025 ]
 [0.068     0.0667   0.045375 ... 0.055475 0.029375 0.262725]]

...

[[0.083525 0.06785  0.044125 ... 0.06365  0.0331   0.234825]
 [0.097825 0.07235  0.047925 ... 0.06675  0.03365  0.2363  ]
 [0.1092   0.082125 0.05385  ... 0.072125 0.036225 0.2486  ]
 ...
 [0.08935  0.088725 0.067575 ... 0.079675 0.042425 0.38085 ]
 [0.093725 0.0875   0.06355  ... 0.07565  0.04185  0.344525]
 [0.0937   0.089675 0.066775 ... 0.07465  0.043025 0.330925]]

[[0.0893  0.0732  0.04715  ... 0.065   0.0351   0.233525]
 [0.091325 0.073425 0.047475 ... 0.0653  0.032675 0.238325]
 [0.096775 0.07645  0.051625 ... 0.06875  0.0344   0.252825]
 ...
 [0.0836   0.084875 0.061975 ... 0.07825  0.042875 0.38785 ]
 [0.08865  0.083825 0.060675 ... 0.0765   0.042525 0.3522  ]
 [0.0909   0.084475 0.061975 ... 0.0769   0.043275 0.342625]]

[[0.092075 0.078   0.050925 ... 0.06565  0.03555  0.235275]
 [0.0805   0.0705  0.043325 ... 0.063925 0.03215  0.243875]
 [0.086925 0.074025 0.0495   ... 0.067475 0.03345  0.26095  ]
 ...
 [0.081075 0.078725 0.056425 ... 0.07505  0.0398   0.37805 ]
 [0.0865   0.079375 0.05845  ... 0.076175 0.0439   0.3619  ]
 [0.0886   0.077775 0.057725 ... 0.076175 0.042825 0.3439  ]]]

[[[0.076525 0.0703  0.04595  ... 0.055225 0.028025 0.25075 ]
 [0.072025 0.0658  0.0446   ... 0.05555  0.02795  0.24755 ]
 [0.0669   0.06225  0.038125 ... 0.05245  0.027125 0.241425]
 ...
 [0.054175 0.050575 0.029475 ... 0.04845  0.022375 0.23045 ]
 [0.05465  0.052375 0.031125 ... 0.04935  0.024375 0.2282  ]
 [0.052525 0.052725 0.029275 ... 0.048325 0.02325  0.229475]]]

```

```

[[0.0784  0.065975 0.0441   ... 0.0594   0.031425 0.241175]
 [0.075475 0.066225 0.044975 ... 0.05505   0.02915   0.2405  ]
 [0.073375 0.063225 0.044475 ... 0.05435   0.029375 0.243575]
 ...
 [0.047325 0.05035  0.027125 ... 0.04535   0.022275 0.2235  ]
 [0.046475 0.051075 0.026425 ... 0.047025 0.021025 0.2348  ]
 [0.04295  0.050275 0.02575  ... 0.044525 0.01955   0.240875]]

[[0.065825 0.0619   0.04045  ... 0.053225 0.026425 0.236775]
 [0.07745  0.062725 0.040725 ... 0.0573   0.030725 0.2439  ]
 [0.075525 0.063775 0.0434   ... 0.05595   0.030125 0.25005  ]
 ...
 [0.046675 0.048325 0.02605  ... 0.0475   0.0219   0.23165  ]
 [0.046825 0.04955  0.026425 ... 0.0471   0.02055   0.243125]
 [0.04435  0.0498   0.0253   ... 0.04675   0.020775 0.239925]]

...

[[0.028025 0.041275 0.01945  ... 0.039375 0.015675 0.22205  ]
 [0.0245   0.040675 0.018025 ... 0.039575 0.016475 0.2187  ]
 [0.02185  0.03435  0.01665  ... 0.034025 0.015   0.20335  ]
 ...
 [0.1155   0.09395  0.0714   ... 0.058625 0.0275   0.335675]
 [0.117225 0.09435  0.0699   ... 0.05885  0.028175 0.34795  ]
 [0.1168   0.093275 0.06865  ... 0.0585   0.02895   0.353275]]

[[0.032025 0.04075  0.020675 ... 0.04025  0.015525 0.2328  ]
 [0.024525 0.038175 0.018025 ... 0.03785  0.015075 0.21255  ]
 [0.0227   0.03625  0.016425 ... 0.035   0.015075 0.204675]
 ...
 [0.11625  0.093825 0.071275 ... 0.058625 0.02685   0.34765  ]
 [0.115325 0.092175 0.06915  ... 0.05855  0.02745   0.3572  ]
 [0.1143   0.091225 0.067325 ... 0.05835  0.028925 0.357825]]

[[0.033325 0.04015  0.0212   ... 0.037875 0.015575 0.220525]
 [0.027225 0.038525 0.01925  ... 0.03625  0.014825 0.207775]
 [0.02625  0.03785  0.01885  ... 0.035675 0.015175 0.209825]
 ...
 [0.1132   0.09225  0.0699   ... 0.057875 0.027175 0.352875]
 [0.1116   0.090575 0.0685   ... 0.0585   0.027325 0.36045  ]
 [0.110325 0.089725 0.06665  ... 0.059425 0.02975   0.35485  ]]]

```

```

[[[0.076325 0.0714 0.0511 ... 0.05685 0.027375 0.3285 ]
 [0.078825 0.066725 0.044825 ... 0.05665 0.03155 0.3196 ]
 [0.1038 0.0806 0.060575 ... 0.07545 0.048225 0.2805 ]
 ...
 [0.02885 0.040825 0.022125 ... 0.037725 0.016275 0.17815 ]
 [0.0286 0.0422 0.02355 ... 0.039625 0.016675 0.191225]
 [0.02775 0.04375 0.022175 ... 0.043325 0.0181 0.203775]]]

[[[0.06785 0.062075 0.04025 ... 0.04975 0.026175 0.31845 ]
 [0.07785 0.06515 0.041575 ... 0.055275 0.033675 0.29555 ]
 [0.099375 0.0823 0.062 ... 0.076125 0.047775 0.27305 ]
 ...
 [0.026425 0.040625 0.021825 ... 0.037175 0.0163 0.180075]
 [0.0283 0.04245 0.02205 ... 0.04045 0.017175 0.192025]
 [0.02925 0.0436 0.022975 ... 0.043725 0.0179 0.20435 ]]]

[[[0.064725 0.0621 0.0413 ... 0.05105 0.02655 0.30515 ]
 [0.08075 0.067625 0.0489 ... 0.0599 0.033625 0.28425 ]
 [0.1018 0.078725 0.060025 ... 0.0735 0.043225 0.2772 ]
 ...
 [0.0277 0.0412 0.020975 ... 0.03765 0.01625 0.184425]
 [0.02835 0.043125 0.021675 ... 0.040175 0.017375 0.19335 ]
 [0.030575 0.043325 0.023375 ... 0.04225 0.0173 0.200575]]]

...

[[[0.06545 0.054525 0.034075 ... 0.05745 0.028325 0.244075]
 [0.06275 0.053075 0.03125 ... 0.055625 0.027675 0.247475]
 [0.060875 0.05235 0.030725 ... 0.053875 0.026575 0.247275]
 ...
 [0.04905 0.0508 0.031375 ... 0.039275 0.018625 0.184025]
 [0.047775 0.04855 0.03135 ... 0.038075 0.017725 0.173025]
 [0.048475 0.052025 0.0336 ... 0.0377 0.018625 0.172875]]]

[[[0.061575 0.051675 0.03085 ... 0.052975 0.02525 0.244675]
 [0.056875 0.050975 0.027025 ... 0.051675 0.023125 0.243075]
 [0.051075 0.05215 0.027025 ... 0.052125 0.022625 0.2422 ]
 ...
 [0.051525 0.05075 0.031625 ... 0.039625 0.021775 0.1806 ]
 [0.0485 0.049475 0.031275 ... 0.03685 0.01885 0.181675]
 [0.054275 0.054875 0.036125 ... 0.037525 0.0198 0.171425]]]

```

```

[[0.055875 0.051075 0.02745 ... 0.04885 0.02285 0.2407 ]
[0.056     0.052725 0.0285 ... 0.053175 0.02415 0.24375 ]
[0.0544    0.05275  0.02815 ... 0.0555   0.0232   0.24885 ]
...
[0.05005   0.051775 0.031 ... 0.03915  0.019525 0.1762  ]
[0.048825  0.051275 0.0324 ... 0.036175 0.018375 0.18395 ]
[0.0513    0.051225 0.031875 ... 0.0385   0.020625 0.177575]]]

...

[[[0.059125 0.0521   0.0284 ... 0.046025 0.019975 0.234825]
 [0.06905   0.055875 0.0304 ... 0.04825   0.021725 0.237375]
 [0.0699    0.05865   0.031125 ... 0.051375 0.022725 0.23655 ]
 ...
 [0.034575  0.04225   0.0247 ... 0.03785   0.019175 0.157225]
 [0.029975  0.038475 0.023925 ... 0.034475 0.014425 0.175175]
 [0.025325  0.03555   0.02115 ... 0.0325    0.0144   0.157   ]]]

[[0.04895   0.051125 0.02935 ... 0.04475   0.0215   0.2242  ]
 [0.0563     0.05555   0.032025 ... 0.04655   0.0231   0.224225]
 [0.055875  0.0564    0.032875 ... 0.04815   0.023    0.232925]
 ...
 [0.0347     0.0392    0.0209 ... 0.035425  0.015675 0.17295 ]
 [0.031875  0.0362     0.02055 ... 0.029625  0.013925 0.14845 ]
 [0.028125  0.03385    0.020825 ... 0.026825  0.01315  0.13235 ]]]

[[0.0486     0.0514    0.028275 ... 0.046925 0.022425 0.22335 ]
 [0.05655    0.053425 0.028925 ... 0.047275 0.022825 0.219525]
 [0.0573     0.055525 0.0294 ... 0.0482    0.022275 0.2325  ]
 ...
 [0.024925  0.0378     0.019675 ... 0.032725 0.01405  0.18365 ]
 [0.031925  0.033875 0.0206 ... 0.03015   0.014075 0.169075]
 [0.0316     0.032025 0.019625 ... 0.0268    0.012925 0.136325]]]

...

[[0.067175 0.0628    0.039875 ... 0.052775 0.0307   0.2282  ]
 [0.080275 0.071475 0.050425 ... 0.0566    0.0342   0.217525]
 [0.07215   0.068375 0.045875 ... 0.056375 0.034375 0.2167  ]
 ...
 [0.03785   0.041425 0.023875 ... 0.043775 0.019575 0.213625]

```

```

[0.03475 0.0394 0.02255 ... 0.04455 0.02 0.217375]
[0.032625 0.039025 0.02305 ... 0.043425 0.01985 0.229575]]

[[[0.07875 0.068475 0.0437 ... 0.056175 0.0339 0.22795 ]
[0.08205 0.073825 0.0498 ... 0.057775 0.035225 0.2253 ]
[0.08115 0.07405 0.0505 ... 0.059475 0.03475 0.2217 ]
...
[0.03895 0.043275 0.026075 ... 0.044775 0.021 0.2286 ]
[0.03795 0.038525 0.02265 ... 0.04295 0.018625 0.22255 ]
[0.03365 0.038425 0.02355 ... 0.042 0.0189 0.225125]]]

[[[0.089 0.076325 0.0531 ... 0.05915 0.0333 0.228925]
[0.084925 0.075775 0.050825 ... 0.05925 0.0363 0.236375]
[0.08475 0.077325 0.050925 ... 0.0591 0.03615 0.225875]
...
[0.040075 0.0416 0.025975 ... 0.044 0.020425 0.234125]
[0.038075 0.036475 0.022375 ... 0.042175 0.01925 0.21895 ]
[0.0349 0.036575 0.0241 ... 0.041525 0.0202 0.223625]]]

[[[0.039875 0.055875 0.031825 ... 0.046725 0.0206 0.2473 ]
[0.041225 0.053475 0.031675 ... 0.04425 0.01995 0.2442 ]
[0.038 0.0509 0.030125 ... 0.04345 0.018975 0.252075]
...
[0.079575 0.068025 0.048175 ... 0.0623 0.0347 0.275575]
[0.093775 0.08395 0.063975 ... 0.12865 0.096575 0.214425]
[0.102475 0.09315 0.07065 ... 0.124725 0.11835 0.17915 ]]

[[[0.039875 0.055025 0.034025 ... 0.0453 0.020225 0.25715 ]
[0.039625 0.053725 0.032925 ... 0.0437 0.01945 0.250625]
[0.03925 0.051775 0.031525 ... 0.0442 0.018825 0.2608 ]
...
[0.080175 0.073025 0.052975 ... 0.06945 0.0391 0.219825]
[0.09105 0.0811 0.05875 ... 0.09675 0.067 0.133375]
[0.08775 0.0791 0.053775 ... 0.097075 0.066325 0.1061 ]]

[[[0.04015 0.05545 0.0358 ... 0.046 0.020325 0.2604 ]
[0.0386 0.053425 0.035075 ... 0.04415 0.0186 0.259075]
[0.038875 0.0541 0.035 ... 0.04585 0.0204 0.2731 ]
...
[0.09545 0.086025 0.06205 ... 0.08275 0.050225 0.117975]
[0.07805 0.07245 0.05015 ... 0.08905 0.06075 0.088825]
[0.075975 0.07035 0.04505 ... 0.09075 0.064575 0.082325]]]

```

...

[[0.041475 0.041475 0.021175 ... 0.03885 0.015775 0.209025]
[0.039625 0.040275 0.021525 ... 0.0381 0.01435 0.199925]
[0.034975 0.040175 0.020375 ... 0.0356 0.014575 0.1891]

...

[0.0552 0.048575 0.034275 ... 0.037725 0.020475 0.150825]
[0.046975 0.04565 0.03075 ... 0.0352 0.01815 0.137475]
[0.049075 0.04705 0.031375 ... 0.03935 0.02075 0.1534]]

[[0.0475 0.04265 0.024375 ... 0.039125 0.0159 0.2042]
[0.048075 0.042075 0.0262 ... 0.039575 0.015975 0.1975]
[0.0455 0.041725 0.02305 ... 0.0391 0.0166 0.203425]

...

[0.054875 0.04825 0.0329 ... 0.036975 0.020325 0.14335]
[0.04635 0.0461 0.0307 ... 0.0349 0.018575 0.1444]
[0.0477 0.045825 0.030225 ... 0.038175 0.0193 0.14945]]

[[0.047625 0.042275 0.025025 ... 0.039375 0.016775 0.2007]
[0.04795 0.043 0.02435 ... 0.039425 0.01655 0.198825]
[0.057725 0.04625 0.03155 ... 0.0416 0.0185 0.20395]

...

[0.0496 0.04615 0.03035 ... 0.036125 0.01925 0.138325]
[0.0501 0.047175 0.030225 ... 0.0391 0.0216 0.158675]
[0.04975 0.048025 0.030475 ... 0.038725 0.021075 0.1527]]

[[[0.09655 0.074775 0.050975 ... 0.0516 0.023025 0.261275]
[0.092725 0.072675 0.0496 ... 0.058225 0.0292 0.208175]
[0.080925 0.064725 0.04845 ... 0.08235 0.050425 0.170475]

...

[0.047575 0.051725 0.026375 ... 0.044925 0.017175 0.256825]
[0.055575 0.052925 0.030125 ... 0.048075 0.018 0.27485]
[0.055525 0.0531 0.0318 ... 0.04635 0.01725 0.256675]]

[[0.095525 0.07545 0.05235 ... 0.053225 0.022625 0.271925]
[0.0957 0.075225 0.05265 ... 0.057725 0.02675 0.219325]
[0.0937 0.071825 0.05245 ... 0.0824 0.05045 0.18085]

...

[0.042775 0.048825 0.02565 ... 0.043875 0.016375 0.257325]
[0.050625 0.051 0.028075 ... 0.04785 0.017925 0.282775]
[0.0558 0.052 0.029675 ... 0.046875 0.017275 0.268275]]

```

[[[0.09525  0.076025 0.0528   ... 0.0533   0.021625 0.2891   ]
  [0.09735  0.0765   0.053    ... 0.055425 0.024675 0.244825]
  [0.09475  0.075125 0.05085   ... 0.071575 0.040575 0.1881   ]
  ...
  [0.038275 0.0477   0.0243   ... 0.043325 0.016    0.2494   ]
  [0.04245  0.050225 0.0255   ... 0.046025 0.01685  0.259525]
  [0.0483   0.052175 0.02775   ... 0.04545  0.017225 0.249375]]]

...

[[[0.033875 0.045775 0.029025 ... 0.0404   0.018975 0.2029   ]
  [0.0357   0.04645  0.028025 ... 0.041925 0.0196   0.20415  ]
  [0.036975 0.046825 0.02825   ... 0.04005  0.018575 0.19235  ]
  ...
  [0.116775 0.0982   0.080175 ... 0.08415  0.06735  0.2857   ]
  [0.104525 0.09055  0.071025 ... 0.0795   0.0627   0.310825]
  [0.0975   0.082025 0.059075 ... 0.06885  0.045825 0.324375]]]

[[[0.035775 0.042825 0.02835   ... 0.039125 0.0173   0.20685  ]
  [0.03505  0.0427   0.028275 ... 0.0397   0.017525 0.2041   ]
  [0.03665  0.0459   0.027125 ... 0.041575 0.0189   0.20055  ]
  ...
  [0.10555  0.088325 0.06645   ... 0.081425 0.059475 0.288725]
  [0.10945  0.091575 0.072325 ... 0.084475 0.057925 0.306175]
  [0.096675 0.0814   0.060425 ... 0.069775 0.04325  0.323975]]]

[[[0.0381   0.0465   0.027175 ... 0.0385   0.0179   0.199175]
  [0.036325 0.04335  0.027625 ... 0.037975 0.016925 0.1999   ]
  [0.036475 0.047725 0.029125 ... 0.043325 0.019775 0.21835  ]
  ...
  [0.1108   0.1004   0.0796   ... 0.0981   0.084725 0.291575]
  [0.0959   0.0824   0.06165  ... 0.07685  0.0528   0.318575]
  [0.093025 0.07815  0.0585   ... 0.06915  0.0448   0.32745  ]]]], shape=(32, 256, 256, 8), dtype=float32)
outputs: float32 (32, 256, 256, 5)
tf.Tensor(
[[[[[0. 0. 1. 0. 0.]
      [0. 0. 1. 0. 0.]
      [0. 0. 1. 0. 0.]
      ...
      [0. 0. 1. 0. 0.]
      [0. 0. 1. 0. 0.]
      [0. 0. 1. 0. 0.]

```

```

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]]

```

```

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]]

```

...

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]

```

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

```

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]]

```



```
[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   ...
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]]]
```

```
[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   ...
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]]]
```

```
[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   ...
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]]]
```

...

```
[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   ...
   [0. 1. 0. 0. 0.]
   [0. 1. 0. 0. 0.]
   [0. 1. 0. 0. 0.]]]
```

```
[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   ...
   [0. 1. 0. 0. 0.]
   [0. 1. 0. 0. 0.]
   [0. 1. 0. 0. 0.]]]
```

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]]

```

```

[[[1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [0. 0. 1. 0. 0.]
   ...
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]]]

```

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

```

```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

```

...

```

[[0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 ...
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]]

```

```

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]]]

...

[[[0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]
   [0. 0. 1. 0. 0.]
   ...
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]
   [1. 0. 0. 0. 0.]]

 [[0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]
  ...
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]]

 [[0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]
  ...
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]]]

```

...

```
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
...
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]]
```

```
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
...
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]]
```

```
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
...
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]
[1. 0. 0. 0. 0.]]]
```

```
[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]]
```

```
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 1. 0. 0.]
...
[1. 0. 0. 0. 0.]
[0. 0. 0. 0. 1.]
[0. 0. 0. 0. 1.]]
```

```

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 1.]]

...

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]]

[[[0. 0. 1. 0. 0.]
  [0. 1. 0. 0. 0.]
  [0. 0. 0. 0. 1.]
  ...
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0.]]]

```

```

[[0. 0. 1. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 1.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

...

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 0. 1. 0.]
 [0. 1. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 0. 1. 0.]
 [0. 1. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 0. 0. 1. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]], shape=(32, 256, 256, 5), dtype=float32)
Testing

```

```

inputs: float32 (1, 256, 256, 8)
tf.Tensor(
[[[0.0853    0.0767    0.052625 ... 0.084725 0.048225 0.266675]
  [0.08645   0.076725 0.05415   ... 0.0815    0.049725 0.256475]
  [0.0881    0.07945   0.05675   ... 0.0833    0.049725 0.267    ]
  ...
  [0.041725  0.046875 0.027925 ... 0.04645   0.019175 0.2598   ]
  [0.03835   0.044725 0.024125 ... 0.04525   0.018175 0.2606   ]
  [0.0354    0.03985   0.021875 ... 0.044     0.017925 0.260925]]

[[0.08945   0.072675 0.047475 ... 0.084925 0.045675 0.253325]
 [0.096     0.07225   0.048375 ... 0.088875 0.049475 0.25065   ]
 [0.10235   0.0735    0.0509    ... 0.088175 0.050675 0.269075]
  ...
 [0.042225  0.0459    0.026575 ... 0.04655   0.01875   0.265025]
 [0.040375  0.044525 0.02595   ... 0.04585   0.0186    0.26045   ]
 [0.03615   0.041075 0.022125 ... 0.044825 0.017775 0.263675]]

[[0.087625  0.0762    0.0522    ... 0.084775 0.0459    0.243175]
 [0.09235   0.07215   0.048425 ... 0.0871    0.04725   0.243725]
 [0.104925  0.074375 0.05205   ... 0.0889    0.048275 0.25105   ]
  ...
 [0.04065   0.041975 0.023275 ... 0.043425 0.018075 0.25435   ]
 [0.0382    0.04225   0.02305   ... 0.0432    0.017725 0.254725]
 [0.037025  0.042925 0.022875 ... 0.046575 0.018425 0.259875]]

...

[[0.074575  0.06      0.03945   ... 0.05635   0.03315   0.198025]
 [0.082     0.06205   0.040675 ... 0.058675 0.033075 0.198625]
 [0.080225  0.06355   0.0416    ... 0.059775 0.03395   0.206025]
  ...
 [0.09965   0.082725 0.06805   ... 0.067325 0.05815   0.27725   ]
 [0.0889    0.0679    0.0468    ... 0.0563    0.034875 0.29495   ]
 [0.07205   0.059575 0.04125   ... 0.05235   0.03185   0.3116    ]]

[[0.0768    0.06205   0.039975 ... 0.058175 0.0334    0.197525]
 [0.0797    0.0638    0.041675 ... 0.060425 0.035925 0.1993    ]
 [0.08345   0.063725 0.04135   ... 0.0606    0.03585   0.2044    ]
  ...
 [0.110425  0.089975 0.071475 ... 0.083225 0.07175   0.261625]
 [0.0995    0.076725 0.053175 ... 0.060975 0.043725 0.29315   ]
 [0.07945   0.06385   0.0462    ... 0.059675 0.038375 0.32095   ]]

```

```

[[0.074075 0.0615 0.0395 ... 0.0591 0.03185 0.200825]
 [0.0771 0.06265 0.040775 ... 0.059825 0.033975 0.204725]
 [0.0835 0.063125 0.0417 ... 0.059825 0.034325 0.2 ...]
 ...
 [0.118575 0.0944 0.070325 ... 0.09795 0.078 0.272 ...]
 [0.11975 0.0899 0.063575 ... 0.077975 0.05495 0.306325]
 [0.0861 0.068475 0.049775 ... 0.063225 0.039625 0.3163 ...]]], shape=(1, 256, 256, 8), dtype=float32)
outputs: float32 (1, 256, 256, 5)
tf.Tensor(
[[[[[1. 0. 0. 0. 0.]
      [0. 0. 1. 0. 0.]
      [0. 1. 0. 0. 0.]
      ...
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]]]

[[[0. 0. 1. 0. 0.]
      [0. 0. 0. 1. 0.]
      [0. 0. 0. 1. 0.]
      ...
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]]]

[[[0. 1. 0. 0. 0.]
      [0. 0. 1. 0. 0.]
      [0. 0. 1. 0. 0.]
      ...
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]]]

...

[[[1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]
      ...
      [0. 0. 1. 0. 0.]
      [1. 0. 0. 0. 0.]
      [1. 0. 0. 0. 0.]]]

```



```

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 ...
 [0. 0. 0. 1. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 ...
 [0. 0. 0. 1. 0.]
 [0. 0. 1. 0. 0.]
 [1. 0. 0. 0. 0.]]], shape=(1, 256, 256, 5), dtype=float32)
Validation
inputs: float32 (1, 256, 256, 8)
tf.Tensor(
[[[0.053275 0.043025 0.0284    ... 0.042575 0.01925  0.2313   ]
  [0.0535    0.04265  0.0293    ... 0.043975 0.0191   0.246425]
  [0.049125 0.042675 0.027125 ... 0.042275 0.019325 0.228225]
  ...
  [0.0724    0.064525 0.044325 ... 0.0504    0.0264    0.202325]
  [0.07395   0.0651    0.04495   ... 0.05235   0.02625   0.211175]
  [0.075975 0.0647     0.04615   ... 0.0523     0.027625 0.2079   ]]

  [[0.053025 0.042325 0.02895   ... 0.041625 0.018475 0.239625]
  [0.051225 0.0413    0.029     ... 0.042     0.018375 0.238775]
  [0.04785   0.04345   0.02785   ... 0.042625 0.019825 0.21835  ]
  ...
  [0.067     0.059125 0.042375 ... 0.049375 0.023475 0.18365  ]
  [0.0679    0.06215  0.042125 ... 0.050825 0.0246    0.197125]
  [0.066575 0.062775 0.041925 ... 0.049875 0.0247    0.199775]]]

  [[0.04975  0.03945  0.0265    ... 0.040925 0.01785  0.243675]
  [0.050625 0.040725 0.027925 ... 0.040825 0.018625 0.236075]
  [0.0546    0.04545  0.029725 ... 0.043575 0.021075 0.20885  ]
  ...
  [0.069075 0.0611    0.0435    ... 0.050075 0.02435  0.186325]
  [0.07345  0.063225 0.0452     ... 0.052325 0.02595  0.19745  ]
  [0.068175 0.06035  0.04155   ... 0.04985  0.023925 0.1912   ]]
```

```

...

[[0.064425 0.062275 0.037175 ... 0.0576 0.027975 0.265325]
 [0.058075 0.059925 0.03495 ... 0.05475 0.02585 0.26375 ]
 [0.040675 0.053675 0.028975 ... 0.0482 0.02065 0.250575]
 ...
 [0.0937 0.09025 0.072 ... 0.0486 0.02375 0.2789 ]
 [0.094125 0.091525 0.072925 ... 0.04795 0.02335 0.273275]
 [0.09135 0.08855 0.067875 ... 0.04985 0.023425 0.282475]]

[[0.063175 0.05715 0.03525 ... 0.054475 0.0265 0.2553 ]
 [0.0581 0.0556 0.032875 ... 0.0511 0.0242 0.246625]
 [0.0396 0.0509 0.027975 ... 0.0464 0.020075 0.23445 ]
 ...
 [0.09535 0.0905 0.076275 ... 0.048725 0.0235 0.289175]
 [0.093725 0.09015 0.0717 ... 0.048325 0.02345 0.279575]
 [0.09145 0.088125 0.068475 ... 0.0493 0.023075 0.290275]]

[[0.04605 0.05285 0.0288 ... 0.048925 0.021625 0.2413 ]
 [0.03955 0.051625 0.028325 ... 0.046975 0.020875 0.2319 ]
 [0.0431 0.052225 0.03135 ... 0.042275 0.02 0.221325]
 ...
 [0.099075 0.085075 0.06545 ... 0.051925 0.02575 0.298475]
 [0.100175 0.08775 0.0678 ... 0.05005 0.024175 0.28905 ]
 [0.09685 0.0912 0.07425 ... 0.049975 0.023375 0.290425]]], shape=(1, 256, 256, 8), dtype=float32)
outputs: float32 (1, 256, 256, 5)
tf.Tensor(
[[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

```

```

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]]

...

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]

[[0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]], shape=(1, 256, 256, 5), dtype=float32)
*****
***** building and compiling model... *****
DERIVE_FEATURES: False
Model: "unet"

```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, None, None, 8)]	0	[]
conv2d (Conv2D)	(None, None, None, 32)	2336	['input_1[0][0]']

batch_normalization (Batch Normalization)	(None, None, None, 32)	128	['conv2d[0][0]']
activation (Activation)	(None, None, None, 32)	0	['batch_normalization[0][0]']
activation_1 (Activation)	(None, None, None, 32)	0	['activation[0][0]']
separable_conv2d (SeparableConv2D)	(None, None, None, 64)	2400	['activation_1[0][0]']
batch_normalization_1 (Batch Normalization)	(None, None, None, 64)	256	['separable_conv2d[0][0]']
activation_2 (Activation)	(None, None, None, 64)	0	['batch_normalization_1[0][0]']
separable_conv2d_1 (SeparableConv2D)	(None, None, None, 64)	4736	['activation_2[0][0]']
batch_normalization_2 (Batch Normalization)	(None, None, None, 64)	256	['separable_conv2d_1[0][0]']
max_pooling2d (MaxPooling2D)	(None, None, None, 64)	0	['batch_normalization_2[0][0]']
conv2d_1 (Conv2D)	(None, None, None, 64)	2112	['activation[0][0]']
add (Add)	(None, None, None, 64)	0	['max_pooling2d[0][0]', 'conv2d_1[0][0]']
activation_3 (Activation)	(None, None, None, 64)	0	['add[0][0]']
separable_conv2d_2 (SeparableConv2D)	(None, None, None, 128)	8896	['activation_3[0][0]']
batch_normalization_3 (Batch Normalization)	(None, None, None, 128)	512	['separable_conv2d_2[0][0]']
activation_4 (Activation)	(None, None, None, 128)	0	['batch_normalization_3[0][0]']
separable_conv2d_3 (SeparableConv2D)	(None, None, None, 128)	17664	['activation_4[0][0]']

bleConv2D)			
batch_normalization_4 (BatchNormalization)	(None, None, None, 128)	512	['separable_conv2d_3[0][0]']
max_pooling2d_1 (MaxPooling2D)	(None, None, None, 128)	0	['batch_normalization_4[0][0]']
conv2d_2 (Conv2D)	(None, None, None, 128)	8320	['add[0][0]']
add_1 (Add)	(None, None, None, 128)	0	['max_pooling2d_1[0][0]', 'conv2d_2[0][0]']
activation_5 (Activation)	(None, None, None, 128)	0	['add_1[0][0]']
separable_conv2d_4 (SeparableConv2D)	(None, None, None, 256)	34176	['activation_5[0][0]']
batch_normalization_5 (BatchNormalization)	(None, None, None, 256)	1024	['separable_conv2d_4[0][0]']
activation_6 (Activation)	(None, None, None, 256)	0	['batch_normalization_5[0][0]']
separable_conv2d_5 (SeparableConv2D)	(None, None, None, 256)	68096	['activation_6[0][0]']
batch_normalization_6 (BatchNormalization)	(None, None, None, 256)	1024	['separable_conv2d_5[0][0]']
max_pooling2d_2 (MaxPooling2D)	(None, None, None, 256)	0	['batch_normalization_6[0][0]']
conv2d_3 (Conv2D)	(None, None, None, 256)	33024	['add_1[0][0]']
add_2 (Add)	(None, None, None, 256)	0	['max_pooling2d_2[0][0]', 'conv2d_3[0][0]']
activation_7 (Activation)	(None, None, None, 256)	0	['add_2[0][0]']
conv2d_transpose (Conv2DTranspose)	(None, None, None, 256)	590080	['activation_7[0][0]']

batch_normalization_7 (BatchNormalization)	(None, None, None, 256)	1024	['conv2d_transpose[0][0]
activation_8 (Activation)	(None, None, None, 256)	0	['batch_normalization_7[
conv2d_transpose_1 (Conv2DTranspose)	(None, None, None, 256)	590080	['activation_8[0][0]']
batch_normalization_8 (BatchNormalization)	(None, None, None, 256)	1024	['conv2d_transpose_1[0][
up_sampling2d_1 (UpSampling2D)	(None, None, None, 256)	0	['add_2[0][0]']
up_sampling2d (UpSampling2D)	(None, None, None, 256)	0	['batch_normalization_8[
conv2d_4 (Conv2D)	(None, None, None, 256)	65792	['up_sampling2d_1[0][0]']
add_3 (Add)	(None, None, None, 256)	0	['up_sampling2d[0][0]', 'conv2d_4[0][0]']
activation_9 (Activation)	(None, None, None, 256)	0	['add_3[0][0]']
conv2d_transpose_2 (Conv2DTranspose)	(None, None, None, 128)	295040	['activation_9[0][0]']
batch_normalization_9 (BatchNormalization)	(None, None, None, 128)	512	['conv2d_transpose_2[0][
activation_10 (Activation)	(None, None, None, 128)	0	['batch_normalization_9[
conv2d_transpose_3 (Conv2DTranspose)	(None, None, None, 128)	147584	['activation_10[0][0]']
batch_normalization_10 (BatchNormalization)	(None, None, None, 128)	512	['conv2d_transpose_3[0][
up_sampling2d_3 (UpSampling2D)	(None, None, None, 256)	0	['add_3[0][0]']

up_sampling2d_2 (UpSampling2D)	(None, None, None, 128)	0	['batch_normalization_10']
conv2d_5 (Conv2D)	(None, None, None, 128)	32896	['up_sampling2d_3[0][0]']
add_4 (Add)	(None, None, None, 128)	0	['up_sampling2d_2[0][0]', 'conv2d_5[0][0]']
activation_11 (Activation)	(None, None, None, 128)	0	['add_4[0][0]']
conv2d_transpose_4 (Conv2D Transpose)	(None, None, None, 64)	73792	['activation_11[0][0]']
batch_normalization_11 (Batch Normalization)	(None, None, None, 64)	256	['conv2d_transpose_4[0][0]']
activation_12 (Activation)	(None, None, None, 64)	0	['batch_normalization_11']
conv2d_transpose_5 (Conv2D Transpose)	(None, None, None, 64)	36928	['activation_12[0][0]']
batch_normalization_12 (Batch Normalization)	(None, None, None, 64)	256	['conv2d_transpose_5[0][0]']
up_sampling2d_5 (UpSampling2D)	(None, None, None, 128)	0	['add_4[0][0]']
up_sampling2d_4 (UpSampling2D)	(None, None, None, 64)	0	['batch_normalization_12']
conv2d_6 (Conv2D)	(None, None, None, 64)	8256	['up_sampling2d_5[0][0]']
add_5 (Add)	(None, None, None, 64)	0	['up_sampling2d_4[0][0]', 'conv2d_6[0][0]']
activation_13 (Activation)	(None, None, None, 64)	0	['add_5[0][0]']
conv2d_transpose_6 (Conv2D Transpose)	(None, None, None, 32)	18464	['activation_13[0][0]']
batch_normalization_13 (Batch Normalization)	(None, None, None, 32)	128	['conv2d_transpose_6[0][0]']

activation_14 (Activation)	(None, None, None, 32)	0	['batch_normalization_13']
conv2d_transpose_7 (Conv2D Transpose)	(None, None, None, 32)	9248	['activation_14[0][0]']
batch_normalization_14 (Batch Normalization)	(None, None, None, 32)	128	['conv2d_transpose_7[0][0]']
up_sampling2d_7 (UpSampling2D)	(None, None, None, 64)	0	['add_5[0][0]']
up_sampling2d_6 (UpSampling2D)	(None, None, None, 32)	0	['batch_normalization_14']
conv2d_7 (Conv2D)	(None, None, None, 32)	2080	['up_sampling2d_7[0][0]']
add_6 (Add)	(None, None, None, 32)	0	['up_sampling2d_6[0][0]', 'conv2d_7[0][0]']
final_conv (Conv2D)	(None, None, None, 5)	1445	['add_6[0][0]']

```

=====
Total params: 2060997 (7.86 MB)
Trainable params: 2057221 (7.85 MB)
Non-trainable params: 3776 (14.75 KB)
-----

```

None

***** preparing output directory... *****

> Saving models and results at /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

***** training model... *****

Epoch 1/30

266/266 [=====] - ETA: 0s - loss: 0.9676 - precision: 0.7271 - recall: 0.7271

Epoch 1: val_loss improved from inf to 2.63403, saving model to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

266/266 [=====] - 299s 971ms/step - loss: 0.9676 - precision: 0.7271 - recall: 0.7271

Epoch 2/30

266/266 [=====] - ETA: 0s - loss: 0.6959 - precision: 0.8066 - recall: 0.8066

Epoch 2: val_loss improved from 2.63403 to 1.23879, saving model to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

266/266 [=====] - 263s 958ms/step - loss: 0.6959 - precision: 0.8066 - recall: 0.8066

Epoch 3/30


```

266/266 [=====] - ETA: 0s - loss: 0.6352 - precision: 0.8285 - recall: 0.8285
Epoch 3: val_loss improved from 1.23879 to 0.62282, saving model to /content/drive/MyDrive/C
266/266 [=====] - 256s 966ms/step - loss: 0.6352 - precision: 0.8285 - recall: 0.8285
Epoch 4/30
266/266 [=====] - ETA: 0s - loss: 0.5988 - precision: 0.8402 - recall: 0.8402
Epoch 4: val_loss improved from 0.62282 to 0.60250, saving model to /content/drive/MyDrive/C
266/266 [=====] - 252s 949ms/step - loss: 0.5988 - precision: 0.8402 - recall: 0.8402
Epoch 5/30
266/266 [=====] - ETA: 0s - loss: 0.5687 - precision: 0.8498 - recall: 0.8498
Epoch 5: val_loss improved from 0.60250 to 0.55160, saving model to /content/drive/MyDrive/C
266/266 [=====] - 285s 1s/step - loss: 0.5687 - precision: 0.8498 - recall: 0.8498
Epoch 6/30
266/266 [=====] - ETA: 0s - loss: 0.5453 - precision: 0.8571 - recall: 0.8571
Epoch 6: val_loss improved from 0.55160 to 0.52872, saving model to /content/drive/MyDrive/C
266/266 [=====] - 284s 1s/step - loss: 0.5453 - precision: 0.8571 - recall: 0.8571
Epoch 7/30
266/266 [=====] - ETA: 0s - loss: 0.5278 - precision: 0.8624 - recall: 0.8624
Epoch 7: val_loss improved from 0.52872 to 0.50506, saving model to /content/drive/MyDrive/C
266/266 [=====] - 262s 988ms/step - loss: 0.5278 - precision: 0.8624 - recall: 0.8624
Epoch 8/30
266/266 [=====] - ETA: 0s - loss: 0.5123 - precision: 0.8671 - recall: 0.8671
Epoch 8: val_loss improved from 0.50506 to 0.49242, saving model to /content/drive/MyDrive/C
266/266 [=====] - 264s 995ms/step - loss: 0.5123 - precision: 0.8671 - recall: 0.8671
Epoch 9/30
266/266 [=====] - ETA: 0s - loss: 0.5018 - precision: 0.8700 - recall: 0.8700
Epoch 9: val_loss did not improve from 0.49242
266/266 [=====] - 276s 1s/step - loss: 0.5018 - precision: 0.8700 - recall: 0.8700
Epoch 10/30
266/266 [=====] - ETA: 0s - loss: 0.4895 - precision: 0.8736 - recall: 0.8736
Epoch 10: val_loss improved from 0.49242 to 0.47591, saving model to /content/drive/MyDrive/C
266/266 [=====] - 268s 1s/step - loss: 0.4895 - precision: 0.8736 - recall: 0.8736
Epoch 11/30
266/266 [=====] - ETA: 0s - loss: 0.4791 - precision: 0.8766 - recall: 0.8766
Epoch 11: val_loss improved from 0.47591 to 0.46856, saving model to /content/drive/MyDrive/C
266/266 [=====] - 263s 992ms/step - loss: 0.4791 - precision: 0.8766 - recall: 0.8766
Epoch 12/30
266/266 [=====] - ETA: 0s - loss: 0.4726 - precision: 0.8783 - recall: 0.8783
Epoch 12: val_loss did not improve from 0.46856
266/266 [=====] - 255s 960ms/step - loss: 0.4726 - precision: 0.8783 - recall: 0.8783
Epoch 13/30
266/266 [=====] - ETA: 0s - loss: 0.4617 - precision: 0.8814 - recall: 0.8814
Epoch 13: val_loss improved from 0.46856 to 0.45125, saving model to /content/drive/MyDrive/C
266/266 [=====] - 266s 1s/step - loss: 0.4617 - precision: 0.8814 - recall: 0.8814

```

Epoch 14/30
 266/266 [=====] - ETA: 0s - loss: 0.4553 - precision: 0.8830 - recall: 0.8830
 Epoch 14: val_loss improved from 0.45125 to 0.44229, saving model to /content/drive/MyDrive/ColabDefaultModel/epoch_14.h5
 266/266 [=====] - 263s 992ms/step - loss: 0.4553 - precision: 0.8830 - recall: 0.8830
 Epoch 15/30
 266/266 [=====] - ETA: 0s - loss: 0.4488 - precision: 0.8847 - recall: 0.8847
 Epoch 15: val_loss did not improve from 0.44229
 266/266 [=====] - 258s 973ms/step - loss: 0.4488 - precision: 0.8847 - recall: 0.8847
 Epoch 16/30
 266/266 [=====] - ETA: 0s - loss: 0.4440 - precision: 0.8859 - recall: 0.8859
 Epoch 16: val_loss did not improve from 0.44229
 266/266 [=====] - 265s 998ms/step - loss: 0.4440 - precision: 0.8859 - recall: 0.8859
 Epoch 17/30
 266/266 [=====] - ETA: 0s - loss: 0.4378 - precision: 0.8875 - recall: 0.8875
 Epoch 17: val_loss did not improve from 0.44229
 266/266 [=====] - 285s 1s/step - loss: 0.4378 - precision: 0.8875 - recall: 0.8875
 Epoch 18/30
 266/266 [=====] - ETA: 0s - loss: 0.4336 - precision: 0.8884 - recall: 0.8884
 Epoch 18: val_loss improved from 0.44229 to 0.42199, saving model to /content/drive/MyDrive/ColabDefaultModel/epoch_18.h5
 266/266 [=====] - 278s 1s/step - loss: 0.4336 - precision: 0.8884 - recall: 0.8884
 Epoch 19/30
 266/266 [=====] - ETA: 0s - loss: 0.4294 - precision: 0.8894 - recall: 0.8894
 Epoch 19: val_loss improved from 0.42199 to 0.41151, saving model to /content/drive/MyDrive/ColabDefaultModel/epoch_19.h5
 266/266 [=====] - 282s 1s/step - loss: 0.4294 - precision: 0.8894 - recall: 0.8894
 Epoch 20/30
 266/266 [=====] - ETA: 0s - loss: 0.4241 - precision: 0.8907 - recall: 0.8907
 Epoch 20: val_loss did not improve from 0.41151
 266/266 [=====] - 258s 970ms/step - loss: 0.4241 - precision: 0.8907 - recall: 0.8907
 Epoch 21/30
 266/266 [=====] - ETA: 0s - loss: 0.4196 - precision: 0.8919 - recall: 0.8919
 Epoch 21: val_loss did not improve from 0.41151
 266/266 [=====] - 259s 977ms/step - loss: 0.4196 - precision: 0.8919 - recall: 0.8919
 Epoch 22/30
 266/266 [=====] - ETA: 0s - loss: 0.4171 - precision: 0.8923 - recall: 0.8923
 Epoch 22: val_loss did not improve from 0.41151
 266/266 [=====] - 260s 982ms/step - loss: 0.4171 - precision: 0.8923 - recall: 0.8923
 Epoch 23/30
 266/266 [=====] - ETA: 0s - loss: 0.4134 - precision: 0.8931 - recall: 0.8931
 Epoch 23: val_loss improved from 0.41151 to 0.40218, saving model to /content/drive/MyDrive/ColabDefaultModel/epoch_23.h5
 266/266 [=====] - 272s 1s/step - loss: 0.4134 - precision: 0.8931 - recall: 0.8931
 Epoch 24/30
 266/266 [=====] - ETA: 0s - loss: 0.4117 - precision: 0.8936 - recall: 0.8936
 Epoch 24: val_loss did not improve from 0.40218

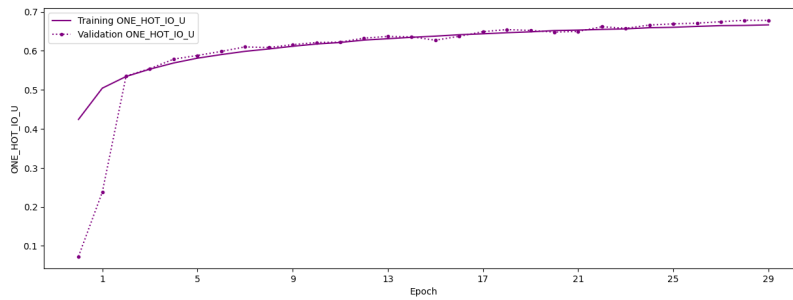
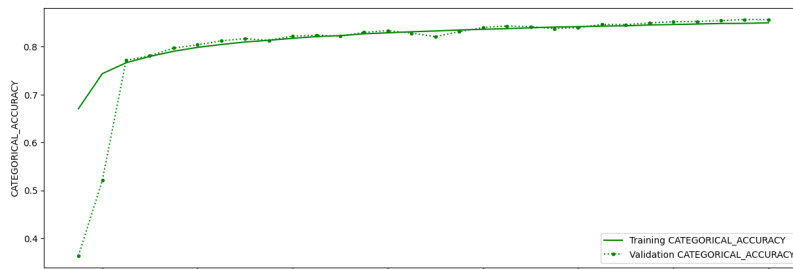
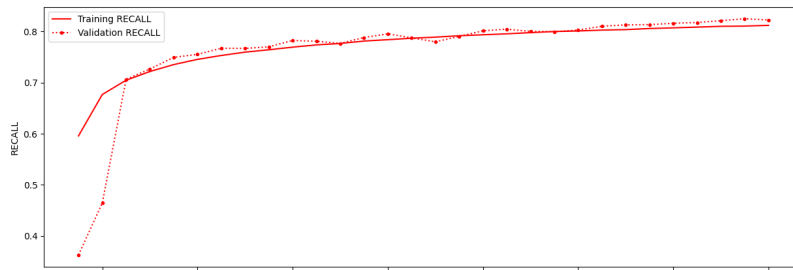
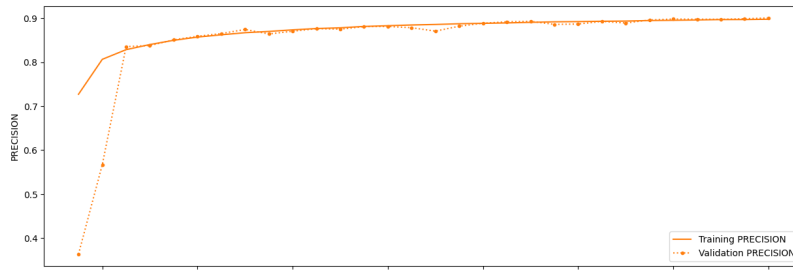
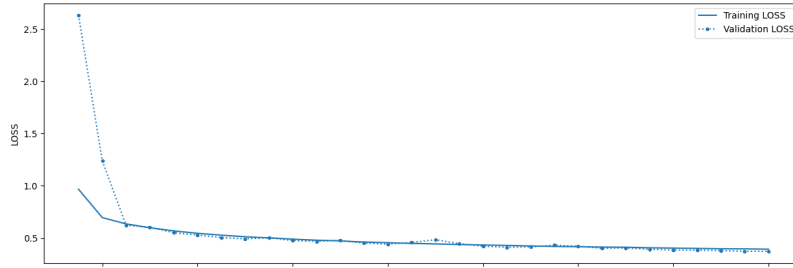
```

266/266 [=====] - 260s 981ms/step - loss: 0.4117 - precision: 0.8936
Epoch 25/30
266/266 [=====] - ETA: 0s - loss: 0.4064 - precision: 0.8948 - recall: 0.8948
Epoch 25: val_loss improved from 0.40218 to 0.39190, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_25_val_loss_improved.h5
266/266 [=====] - 298s 1s/step - loss: 0.4064 - precision: 0.8948 - recall: 0.8948
Epoch 26/30
266/266 [=====] - ETA: 0s - loss: 0.4038 - precision: 0.8954 - recall: 0.8954
Epoch 26: val_loss improved from 0.39190 to 0.38542, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_26_val_loss_improved.h5
266/266 [=====] - 280s 1s/step - loss: 0.4038 - precision: 0.8954 - recall: 0.8954
Epoch 27/30
266/266 [=====] - ETA: 0s - loss: 0.4007 - precision: 0.8961 - recall: 0.8961
Epoch 27: val_loss improved from 0.38542 to 0.38379, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_27_val_loss_improved.h5
266/266 [=====] - 283s 1s/step - loss: 0.4007 - precision: 0.8961 - recall: 0.8961
Epoch 28/30
266/266 [=====] - ETA: 0s - loss: 0.3972 - precision: 0.8969 - recall: 0.8969
Epoch 28: val_loss improved from 0.38379 to 0.37968, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_28_val_loss_improved.h5
266/266 [=====] - 264s 996ms/step - loss: 0.3972 - precision: 0.8969 - recall: 0.8969
Epoch 29/30
266/266 [=====] - ETA: 0s - loss: 0.3962 - precision: 0.8970 - recall: 0.8970
Epoch 29: val_loss improved from 0.37968 to 0.37328, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_29_val_loss_improved.h5
266/266 [=====] - 263s 989ms/step - loss: 0.3962 - precision: 0.8970 - recall: 0.8970
Epoch 30/30
266/266 [=====] - ETA: 0s - loss: 0.3934 - precision: 0.8977 - recall: 0.8977
Epoch 30: val_loss improved from 0.37328 to 0.37270, saving model to /content/drive/MyDrive/colab_default_drive_content/epoch_30_val_loss_improved.h5
266/266 [=====] - 265s 998ms/step - loss: 0.3934 - precision: 0.8977 - recall: 0.8977
*****
***** evaluating model... *****
*****
*****
Validation
1222/1222 [=====] - 36s 22ms/step - loss: 0.3723 - precision: 0.9008 - recall: 0.9008
loss: 0.3722515106201172
precision: 0.9008649587631226
recall: 0.8225432634353638
categorical_accuracy: 0.8566350340843201
one_hot_io_u: 0.6780571341514587

*****
***** saving parameters... *****
*****
***** saving model config and history object... *****
*****

```

```
***** saving plots... *****
Saving plots and model visualization at /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapt
*****
***** saving models... *****
*****
```



1.4.4 Save the config file

```
from pathlib import Path
import shutil

config_file = Path(config_file)
drive_config_file = Path(unet_config.MODEL_DIR / f"{str(config_file).split('/')[1]}")

# Create the target directory if it doesn't exist
drive_config_file.parent.mkdir(parents=True, exist_ok=True)

# Copy the file
shutil.copy(config_file, drive_config_file)

print(f"File copied from {config_file} to {drive_config_file}")
```

File copied from servir-aces/config.env to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output/unet_v1/logs'

1.4.5 Load the logs files via TensorBoard

Tensorboard provides a unique way to view and interact with the logs while the model is being trained. Learn more [here](#). Here we only show you how you can load them to tensorboard with our training logs.

```
# Load the TensorBoard notebook extension
%load_ext tensorboard
```

```
log_dir_unet = f"{str(unet_config.MODEL_DIR)}/logs"
log_dir_unet
```

'/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output/unet_v1/logs'

```
%tensorboard --logdir "{log_dir_unet}"
```

Reusing TensorBoard on port 6007 (pid 5630), started 0:02:00 ago. (Use '!kill 5630' to kill)

<IPython.core.display.Javascript object>

1.4.6 Load the Saved U-Net Model

Load the saved model

```
import tensorflow as tf

unet_model = tf.keras.models.load_model(f"{str(unet_config.MODEL_DIR)}/trained-model")

print(unet_model.summary())
```

Model: "unet"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, None, None, 8)]	0	[]
conv2d (Conv2D)	(None, None, None, 32)	2336	['input_1[0][0]']
batch_normalization (Batch Normalization)	(None, None, None, 32)	128	['conv2d[0][0]']
activation (Activation)	(None, None, None, 32)	0	['batch_normalization[0]
activation_1 (Activation)	(None, None, None, 32)	0	['activation[0][0]']
separable_conv2d (SeparableConv2D)	(None, None, None, 64)	2400	['activation_1[0][0]']
batch_normalization_1 (Batch Normalization)	(None, None, None, 64)	256	['separable_conv2d[0][0]
activation_2 (Activation)	(None, None, None, 64)	0	['batch_normalization_1[
separable_conv2d_1 (SeparableConv2D)	(None, None, None, 64)	4736	['activation_2[0][0]']
batch_normalization_2 (Batch Normalization)	(None, None, None, 64)	256	['separable_conv2d_1[0][
max_pooling2d (MaxPooling2D)	(None, None, None, 64)	0	['batch_normalization_2[

D)			]
conv2d_1 (Conv2D)	(None, None, None, 64)	2112	['activation[0][0]']
add (Add)	(None, None, None, 64)	0	['max_pooling2d[0][0]', 'conv2d_1[0][0]']
activation_3 (Activation)	(None, None, None, 64)	0	['add[0][0]']
separable_conv2d_2 (SeparableConv2D)	(None, None, None, 128)	8896	['activation_3[0][0]']
batch_normalization_3 (BatchNormalization)	(None, None, None, 128)	512	['separable_conv2d_2[0][0]']
activation_4 (Activation)	(None, None, None, 128)	0	['batch_normalization_3[0][0]']
separable_conv2d_3 (SeparableConv2D)	(None, None, None, 128)	17664	['activation_4[0][0]']
batch_normalization_4 (BatchNormalization)	(None, None, None, 128)	512	['separable_conv2d_3[0][0]']
max_pooling2d_1 (MaxPooling2D)	(None, None, None, 128)	0	['batch_normalization_4[0][0]']
conv2d_2 (Conv2D)	(None, None, None, 128)	8320	['add[0][0]']
add_1 (Add)	(None, None, None, 128)	0	['max_pooling2d_1[0][0]', 'conv2d_2[0][0]']
activation_5 (Activation)	(None, None, None, 128)	0	['add_1[0][0]']
separable_conv2d_4 (SeparableConv2D)	(None, None, None, 256)	34176	['activation_5[0][0]']
batch_normalization_5 (BatchNormalization)	(None, None, None, 256)	1024	['separable_conv2d_4[0][0]']
activation_6 (Activation)	(None, None, None, 256)	0	['batch_normalization_5[0][0]']

separable_conv2d_5 (SeparableConv2D)	(None, None, None, 256)	68096	['activation_6[0][0]']
batch_normalization_6 (BatchNormalization)	(None, None, None, 256)	1024	['separable_conv2d_5[0][0]']
max_pooling2d_2 (MaxPooling2D)	(None, None, None, 256)	0	['batch_normalization_6[0][0]']
conv2d_3 (Conv2D)	(None, None, None, 256)	33024	['add_1[0][0]']
add_2 (Add)	(None, None, None, 256)	0	['max_pooling2d_2[0][0]', 'conv2d_3[0][0]']
activation_7 (Activation)	(None, None, None, 256)	0	['add_2[0][0]']
conv2d_transpose (Conv2DTranspose)	(None, None, None, 256)	590080	['activation_7[0][0]']
batch_normalization_7 (BatchNormalization)	(None, None, None, 256)	1024	['conv2d_transpose[0][0]']
activation_8 (Activation)	(None, None, None, 256)	0	['batch_normalization_7[0][0]']
conv2d_transpose_1 (Conv2DTranspose)	(None, None, None, 256)	590080	['activation_8[0][0]']
batch_normalization_8 (BatchNormalization)	(None, None, None, 256)	1024	['conv2d_transpose_1[0][0]']
up_sampling2d_1 (UpSampling2D)	(None, None, None, 256)	0	['add_2[0][0]']
up_sampling2d (UpSampling2D)	(None, None, None, 256)	0	['batch_normalization_8[0][0]']
conv2d_4 (Conv2D)	(None, None, None, 256)	65792	['up_sampling2d_1[0][0]']
add_3 (Add)	(None, None, None, 256)	0	['up_sampling2d[0][0]', 'conv2d_4[0][0]']
activation_9 (Activation)	(None, None, None, 256)	0	['add_3[0][0]']

conv2d_transpose_2 (Conv2D Transpose)	(None, None, None, 128)	295040	['activation_9[0][0]']
batch_normalization_9 (BatchNormalization)	(None, None, None, 128)	512	['conv2d_transpose_2[0][0]']
activation_10 (Activation)	(None, None, None, 128)	0	['batch_normalization_9[0][0]']
conv2d_transpose_3 (Conv2D Transpose)	(None, None, None, 128)	147584	['activation_10[0][0]']
batch_normalization_10 (BatchNormalization)	(None, None, None, 128)	512	['conv2d_transpose_3[0][0]']
up_sampling2d_3 (UpSampling2D)	(None, None, None, 256)	0	['add_3[0][0]']
up_sampling2d_2 (UpSampling2D)	(None, None, None, 128)	0	['batch_normalization_10[0][0]']
conv2d_5 (Conv2D)	(None, None, None, 128)	32896	['up_sampling2d_3[0][0]']
add_4 (Add)	(None, None, None, 128)	0	['up_sampling2d_2[0][0]'] ['conv2d_5[0][0]']
activation_11 (Activation)	(None, None, None, 128)	0	['add_4[0][0]']
conv2d_transpose_4 (Conv2D Transpose)	(None, None, None, 64)	73792	['activation_11[0][0]']
batch_normalization_11 (BatchNormalization)	(None, None, None, 64)	256	['conv2d_transpose_4[0][0]']
activation_12 (Activation)	(None, None, None, 64)	0	['batch_normalization_11[0][0]']
conv2d_transpose_5 (Conv2D Transpose)	(None, None, None, 64)	36928	['activation_12[0][0]']
batch_normalization_12 (BatchNormalization)	(None, None, None, 64)	256	['conv2d_transpose_5[0][0]']

up_sampling2d_5 (UpSampling2D)	(None, None, None, 128)	0	['add_4[0][0]']
up_sampling2d_4 (UpSampling2D)	(None, None, None, 64)	0	['batch_normalization_12']
conv2d_6 (Conv2D)	(None, None, None, 64)	8256	['up_sampling2d_5[0][0]']
add_5 (Add)	(None, None, None, 64)	0	['up_sampling2d_4[0][0]', 'conv2d_6[0][0]']
activation_13 (Activation)	(None, None, None, 64)	0	['add_5[0][0]']
conv2d_transpose_6 (Conv2DTranspose)	(None, None, None, 32)	18464	['activation_13[0][0]']
batch_normalization_13 (BatchNormalization)	(None, None, None, 32)	128	['conv2d_transpose_6[0][0]']
activation_14 (Activation)	(None, None, None, 32)	0	['batch_normalization_13']
conv2d_transpose_7 (Conv2DTranspose)	(None, None, None, 32)	9248	['activation_14[0][0]']
batch_normalization_14 (BatchNormalization)	(None, None, None, 32)	128	['conv2d_transpose_7[0][0]']
up_sampling2d_7 (UpSampling2D)	(None, None, None, 64)	0	['add_5[0][0]']
up_sampling2d_6 (UpSampling2D)	(None, None, None, 32)	0	['batch_normalization_14']
conv2d_7 (Conv2D)	(None, None, None, 32)	2080	['up_sampling2d_7[0][0]']
add_6 (Add)	(None, None, None, 32)	0	['up_sampling2d_6[0][0]', 'conv2d_7[0][0]']
final_conv (Conv2D)	(None, None, None, 5)	1445	['add_6[0][0]']

=====

Total params: 2060997 (7.86 MB)
Trainable params: 2057221 (7.85 MB)
Non-trainable params: 3776 (14.75 KB)

None

1.4.7 Inference using Saved U-Net Model

Now we can use the saved model to start the export of the prediction of the image. For prediction, you would need to first prepare your image data. We have already exported the image needed here, which we will use for now. See [this notebook](#) to understand how we did it.

In addition, [this notebook](#) shows how you can then use the image to predict from the saved Model.

In any case, you now have the prediction in the Earth Engine as image.

1.5 DNN Model

1.5.1 Setup any changes in the config file for DNN Model

There are few config variables that needs to be changed for running a DNN model. First would be the data itself so let's change the DATADIR. We also need to change our output directory using MODEL_DIR_NAME. This is the sub-directory inside the OUTPUT_DIR for this model run. We also need to specify this is the DNN model that we want to run. We have MODEL_TYPE parameter for that. Currently, it supports unet, dnn, and cnn (case sensitive) models; default being unet. Make other changes, as appropriate.

```
DATADIR = "datasets/dnn_planet_wo_indices"  
MODEL_DIR_NAME = "dnn_v1"  
MODEL_TYPE = "dnn"
```

1.5.2 Update the config file programtically

```
DATADIR = "datasets/dnn_planet_wo_indices" # @param {type:"string"}  
# PATCH_SHAPE, USE_ELEVATION, USE_S1, TRAIN_SIZE, TEST_SIZE, VAL_SIZE  
# BATCH_SIZE, EPOCHS are converted to their appropriate type.  
MODEL_DIR_NAME = "dnn_v1" # @param {type:"string"}
```

```
MODEL_TYPE = "dnn" # @param {type:"string"}
BATCH_SIZE = "32" # @param {type:"string"}
EPOCHS = "30" # @param {type:"string"}
```

```
dnn_config_settings = {
    "DATADIR": DATADIR,
    "MODEL_DIR_NAME": MODEL_DIR_NAME,
    "MODEL_TYPE": MODEL_TYPE,
    "BATCH_SIZE": BATCH_SIZE,
    "EPOCHS": EPOCHS,
}
```

```
for config_key in dnn_config_settings:
    dotenv.set_key(dotenv_path=config_file,
                   key_to_set=config_key,
                   value_to_set=dnn_config_settings[config_key]
                   )
```

1.5.3 Load config file variables for DNN Model

```
dnn_config = Config(config_file=config_file, override=True)
```

```
BASEDIR: /content
DATADIR: /content/datasets/dnn_planet_wo_indices
using features: ['red_before', 'green_before', 'blue_before', 'nir_before', 'red_during', 'green_during']
using labels: ['class']
```

Most of the config in the `config.env` is now available via the config instance. Let's check few of them here.

```
dnn_config.TRAINING_DIR, dnn_config.OUTPUT_DIR, dnn_config.BATCH_SIZE, dnn_config.MODEL_TYPE
```

```
(PosixPath('/content/datasets/dnn_planet_wo_indices/training'),
 PosixPath('/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output'),
 32,
 'dnn')
```

1.5.4 Load ModelTrainer class

Next, let's make an instance of the `ModelTrainer` object. The `ModelTrainer` class provides various tools for training, buidling, compiling, and running specified deep learning models.

```
dnn_model_trainer = ModelTrainer(dnn_config, seed=42)
```

Using seed: 42

1.5.5 Train and Save DNN model

```
dnn_model_trainer.train_model()
```

```
*****
***** Clear Session... *****
*****
***** Configure memory growth... *****
> Found 1 GPUs
*****
***** creating datasets... *****
Loading dataset from /content/datasets/dnn_planet_wo_indices/training/*
Loading dataset from /content/datasets/dnn_planet_wo_indices/validation/*
Loading dataset from /content/datasets/dnn_planet_wo_indices/testing/*
Printing dataset info:
Training
inputs: float32 (32, 1, 8)
tf.Tensor(
[[[0.06445  0.0383  0.09815  0.06755  0.269975 0.207325 0.11135
    0.060025]]

 [[0.075925 0.02705  0.08695  0.054775 0.235575 0.291625 0.1049
    0.0364  ]]

 [[0.043625 0.025    0.064175 0.04265  0.22      0.225    0.062025
    0.03195  ]]

 [[0.07915  0.05365  0.1054   0.093425 0.257325 0.28345  0.1119
    0.079675]]

 [[0.06945  0.025825 0.10755  0.062125 0.245125 0.28365  0.116975
```

0.0485]]

[[0.092425 0.07485 0.10285 0.09645 0.238575 0.252075 0.123025
0.094825]]

[[0.0555 0.02955 0.087325 0.09095 0.230625 0.298175 0.08075
0.0524]]

[[0.0643 0.021275 0.085875 0.0431 0.248075 0.27395 0.098625
0.0284]]

[[0.0747 0.047675 0.094625 0.072125 0.253125 0.267225 0.09725
0.063825]]

[[0.0626 0.023575 0.083675 0.04705 0.20645 0.2539 0.10465
0.0317]]

[[0.072725 0.030975 0.10185 0.0666 0.26585 0.38635 0.11875
0.045725]]

[[0.0713 0.0289 0.09355 0.06415 0.236675 0.2564 0.10215
0.049475]]

[[0.086175 0.077875 0.10745 0.0709 0.263475 0.289175 0.1188
0.075975]]

[[0.079575 0.027525 0.102325 0.054775 0.253875 0.2761 0.111325
0.03985]]

[[0.08825 0.0803 0.1016 0.0862 0.265025 0.260925 0.115625
0.092325]]

[[0.08025 0.10325 0.1034 0.13695 0.2283 0.246475 0.109975
0.124]]

[[0.077775 0.029875 0.0953 0.0546 0.235325 0.266 0.122125
0.0469]]

[[0.0778 0.024025 0.103525 0.053975 0.23395 0.263625 0.116025
0.036825]]

[[0.089475 0.070675 0.10515 0.09125 0.257725 0.254375 0.132675
0.09745]]

```

[[0.0785  0.026275 0.105575 0.051025 0.2552   0.2522   0.120125
  0.034825]]

[[0.07945  0.045775 0.094475 0.0652   0.264175 0.335825 0.1147
  0.059275]]

[[0.04725  0.031125 0.08165  0.065325 0.23025  0.2299   0.0981
  0.060525]]

[[0.07475  0.02425  0.10205  0.04945  0.263225 0.178625 0.112775
  0.0373  ]]

[[0.079875 0.0259   0.105875 0.05245  0.2505   0.269425 0.12245
  0.034975]]

[[0.0746   0.033975 0.104075 0.0598   0.25375  0.29345  0.117325
  0.04885  ]]

[[0.067125 0.026625 0.093575 0.05095  0.255925 0.231575 0.10545
  0.0415  ]]

[[0.076325 0.02685  0.10615  0.05585  0.25805  0.276325 0.116725
  0.0417  ]]

[[0.061275 0.023975 0.086025 0.041375 0.199425 0.255125 0.111
  0.030825]]

[[0.059725 0.0203   0.0877   0.0434   0.230125 0.251975 0.105675
  0.027225]]

[[0.07205  0.02285  0.094025 0.04865  0.2084   0.247225 0.119175
  0.03225  ]]

[[0.0405   0.023025 0.06625  0.0505   0.218225 0.258275 0.0802
  0.03875  ]]

[[0.07005  0.023125 0.111925 0.0475   0.2716   0.2495   0.124725
  0.0313  ]], shape=(32, 1, 8), dtype=float32)
outputs: float32 (32, 1, 5)
tf.Tensor(
[[[0. 1. 0. 0. 0.]]

```


[[0. 1. 0. 0. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 1. 0. 0. 0.]]
[[1. 0. 0. 0. 0.]]
[[0. 0. 1. 0. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 0. 0. 1. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 1. 0. 0. 0.]]
[[0. 0. 1. 0. 0.]]
[[0. 1. 0. 0. 0.]]

```

[[0. 1. 0. 0. 0.]]

[[0. 1. 0. 0. 0.]]

[[0. 1. 0. 0. 0.]]

[[0. 1. 0. 0. 0.]]

[[0. 1. 0. 0. 0.]]

[[0. 1. 0. 0. 0.]]

[[0. 0. 0. 0. 1.]]

[[0. 0. 1. 0. 0.]]

[[0. 1. 0. 0. 0.]], shape=(32, 1, 5), dtype=float32)
Testing
inputs: float32 (1, 1, 8)
tf.Tensor(
[[[0.06205  0.0342  0.081075 0.0639  0.24245  0.251675 0.086575
      0.054175]]], shape=(1, 1, 8), dtype=float32)
outputs: float32 (1, 1, 5)
tf.Tensor([[[1. 0. 0. 0. 0.]]], shape=(1, 1, 5), dtype=float32)
Validation
inputs: float32 (1, 1, 8)
tf.Tensor(
[[[0.067225 0.031725 0.092275 0.07245  0.23165  0.2267  0.103025
      0.05155 ]]], shape=(1, 1, 8), dtype=float32)
outputs: float32 (1, 1, 5)
tf.Tensor([[[0. 0. 0. 1. 0.]]], shape=(1, 1, 5), dtype=float32)
*****
***** building and compiling model... *****
INITIAL_BIAS: None
Model: "model"

```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	[(None, None, 8)]	0
dense (Dense)	(None, None, 256)	2304

dropout (Dropout)	(None, None, 256)	0
dense_1 (Dense)	(None, None, 128)	32896
dropout_1 (Dropout)	(None, None, 128)	0
dense_2 (Dense)	(None, None, 64)	8256
dropout_2 (Dropout)	(None, None, 64)	0
dense_3 (Dense)	(None, None, 32)	2080
dropout_3 (Dropout)	(None, None, 32)	0
dense_4 (Dense)	(None, None, 5)	165

=====

Total params: 45701 (178.52 KB)

Trainable params: 45701 (178.52 KB)

Non-trainable params: 0 (0.00 Byte)

None

***** preparing output directory... *****

> Saving models and results at /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

***** training model... *****

Epoch 1/30

264/266 [=====>.] - ETA: 0s - loss: 1.2595 - precision: 0.6700 - recall: 0.6700

Epoch 1: val_loss improved from inf to 1.15955, saving model to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

266/266 [=====] - 31s 99ms/step - loss: 1.2623 - precision: 0.6685 - recall: 0.6685

Epoch 2/30

265/266 [=====>.] - ETA: 0s - loss: 0.9196 - precision: 0.7593 - recall: 0.7593

Epoch 2: val_loss improved from 1.15955 to 0.87784, saving model to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

266/266 [=====] - 16s 60ms/step - loss: 0.9206 - precision: 0.7592 - recall: 0.7592

Epoch 3/30

266/266 [=====] - ETA: 0s - loss: 0.8111 - precision: 0.7857 - recall: 0.7857

Epoch 3: val_loss did not improve from 0.87784

266/266 [=====] - 23s 89ms/step - loss: 0.8111 - precision: 0.7857 - recall: 0.7857

Epoch 4/30

261/266 [=====>.] - ETA: 0s - loss: 0.7535 - precision: 0.8003 - recall: 0.8003

Epoch 4: val_loss improved from 0.87784 to 0.82494, saving model to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output

266/266 [=====] - 16s 61ms/step - loss: 0.7584 - precision: 0.7999 - recall: 0.7999

Epoch 5/30
265/266 [=====>.] - ETA: 0s - loss: 0.7328 - precision: 0.8096 - recall: 0.8096
Epoch 5: val_loss improved from 0.82494 to 0.74722, saving model to /content/drive/MyDrive/C
266/266 [=====] - 18s 66ms/step - loss: 0.7328 - precision: 0.8096 - recall: 0.8096
Epoch 6/30
264/266 [=====>.] - ETA: 0s - loss: 0.7254 - precision: 0.8092 - recall: 0.8092
Epoch 6: val_loss improved from 0.74722 to 0.69727, saving model to /content/drive/MyDrive/C
266/266 [=====] - 16s 61ms/step - loss: 0.7268 - precision: 0.8086 - recall: 0.8086
Epoch 7/30
265/266 [=====>.] - ETA: 0s - loss: 0.7175 - precision: 0.8121 - recall: 0.8121
Epoch 7: val_loss improved from 0.69727 to 0.69563, saving model to /content/drive/MyDrive/C
266/266 [=====] - 16s 61ms/step - loss: 0.7175 - precision: 0.8121 - recall: 0.8121
Epoch 8/30
265/266 [=====>.] - ETA: 0s - loss: 0.7065 - precision: 0.8111 - recall: 0.8111
Epoch 8: val_loss improved from 0.69563 to 0.67199, saving model to /content/drive/MyDrive/C
266/266 [=====] - 24s 92ms/step - loss: 0.7066 - precision: 0.8109 - recall: 0.8109
Epoch 9/30
266/266 [=====] - ETA: 0s - loss: 0.6927 - precision: 0.8161 - recall: 0.8161
Epoch 9: val_loss improved from 0.67199 to 0.66129, saving model to /content/drive/MyDrive/C
266/266 [=====] - 24s 92ms/step - loss: 0.6927 - precision: 0.8161 - recall: 0.8161
Epoch 10/30
264/266 [=====>.] - ETA: 0s - loss: 0.6922 - precision: 0.8143 - recall: 0.8143
Epoch 10: val_loss did not improve from 0.66129
266/266 [=====] - 23s 87ms/step - loss: 0.6936 - precision: 0.8139 - recall: 0.8139
Epoch 11/30
263/266 [=====>.] - ETA: 0s - loss: 0.6978 - precision: 0.8120 - recall: 0.8120
Epoch 11: val_loss did not improve from 0.66129
266/266 [=====] - 23s 87ms/step - loss: 0.6984 - precision: 0.8117 - recall: 0.8117
Epoch 12/30
263/266 [=====>.] - ETA: 0s - loss: 0.6804 - precision: 0.8173 - recall: 0.8173
Epoch 12: val_loss improved from 0.66129 to 0.65464, saving model to /content/drive/MyDrive/C
266/266 [=====] - 17s 63ms/step - loss: 0.6806 - precision: 0.8177 - recall: 0.8177
Epoch 13/30
261/266 [=====>.] - ETA: 0s - loss: 0.6858 - precision: 0.8154 - recall: 0.8154
Epoch 13: val_loss did not improve from 0.65464
266/266 [=====] - 15s 56ms/step - loss: 0.6863 - precision: 0.8152 - recall: 0.8152
Epoch 14/30
264/266 [=====>.] - ETA: 0s - loss: 0.6880 - precision: 0.8123 - recall: 0.8123
Epoch 14: val_loss did not improve from 0.65464
266/266 [=====] - 24s 90ms/step - loss: 0.6894 - precision: 0.8119 - recall: 0.8119
Epoch 15/30
263/266 [=====>.] - ETA: 0s - loss: 0.6748 - precision: 0.8167 - recall: 0.8167
Epoch 15: val_loss improved from 0.65464 to 0.64675, saving model to /content/drive/MyDrive/C

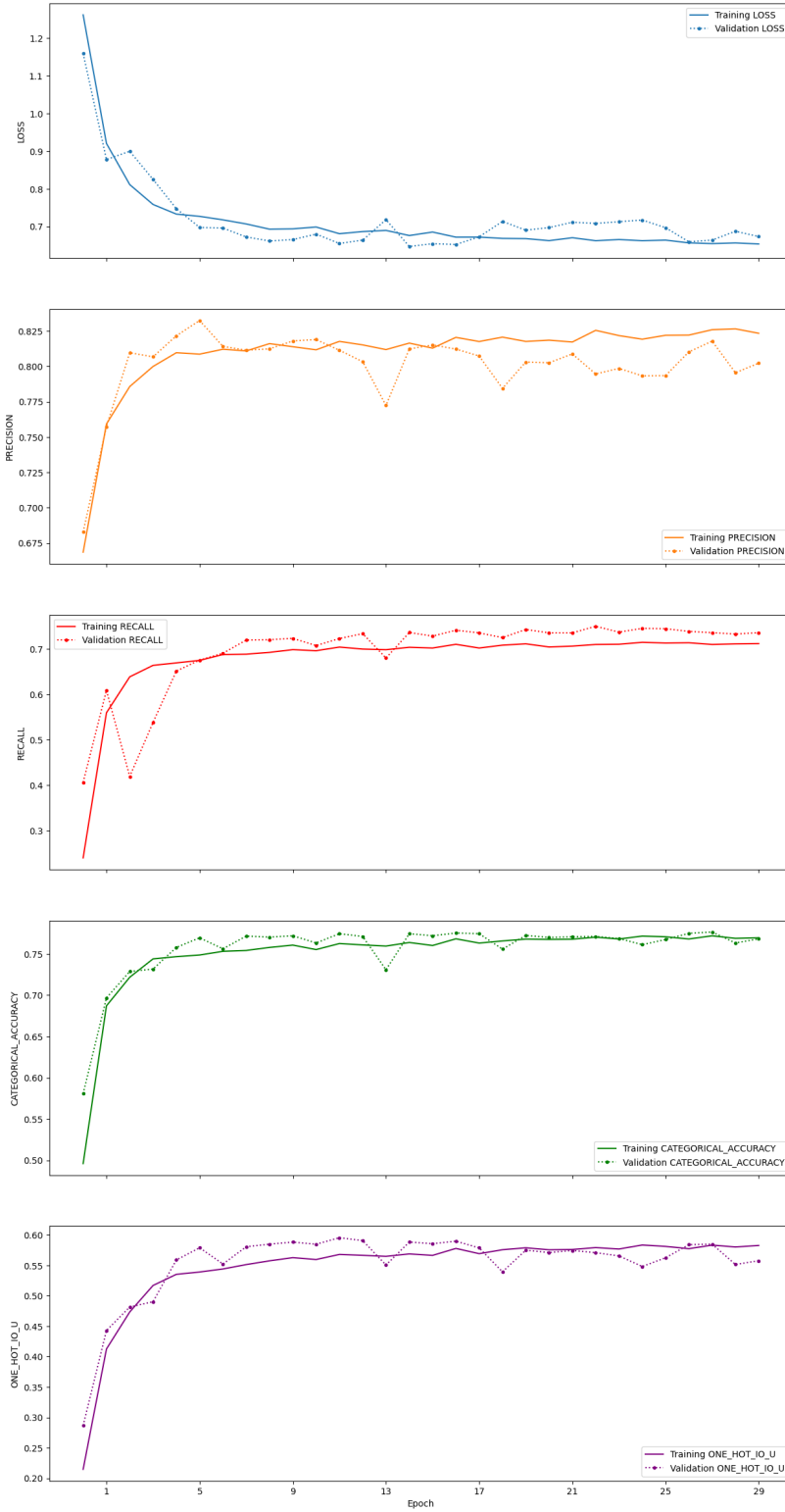
266/266 [=====] - 16s 60ms/step - loss: 0.6758 - precision: 0.8164 - recall: 0.8164
Epoch 16/30
265/266 [=====>.] - ETA: 0s - loss: 0.6859 - precision: 0.8126 - recall: 0.8126
Epoch 16: val_loss did not improve from 0.64675
266/266 [=====] - 24s 90ms/step - loss: 0.6850 - precision: 0.8129 - recall: 0.8129
Epoch 17/30
264/266 [=====>.] - ETA: 0s - loss: 0.6713 - precision: 0.8203 - recall: 0.8203
Epoch 17: val_loss did not improve from 0.64675
266/266 [=====] - 14s 54ms/step - loss: 0.6715 - precision: 0.8205 - recall: 0.8205
Epoch 18/30
261/266 [=====>.] - ETA: 0s - loss: 0.6766 - precision: 0.8158 - recall: 0.8158
Epoch 18: val_loss did not improve from 0.64675
266/266 [=====] - 24s 91ms/step - loss: 0.6716 - precision: 0.8175 - recall: 0.8175
Epoch 19/30
266/266 [=====] - ETA: 0s - loss: 0.6681 - precision: 0.8207 - recall: 0.8207
Epoch 19: val_loss did not improve from 0.64675
266/266 [=====] - 14s 54ms/step - loss: 0.6681 - precision: 0.8207 - recall: 0.8207
Epoch 20/30
263/266 [=====>.] - ETA: 0s - loss: 0.6686 - precision: 0.8171 - recall: 0.8171
Epoch 20: val_loss did not improve from 0.64675
266/266 [=====] - 15s 57ms/step - loss: 0.6676 - precision: 0.8176 - recall: 0.8176
Epoch 21/30
262/266 [=====>.] - ETA: 0s - loss: 0.6652 - precision: 0.8173 - recall: 0.8173
Epoch 21: val_loss did not improve from 0.64675
266/266 [=====] - 23s 89ms/step - loss: 0.6622 - precision: 0.8185 - recall: 0.8185
Epoch 22/30
265/266 [=====>.] - ETA: 0s - loss: 0.6699 - precision: 0.8170 - recall: 0.8170
Epoch 22: val_loss did not improve from 0.64675
266/266 [=====] - 17s 63ms/step - loss: 0.6699 - precision: 0.8172 - recall: 0.8172
Epoch 23/30
262/266 [=====>.] - ETA: 0s - loss: 0.6639 - precision: 0.8248 - recall: 0.8248
Epoch 23: val_loss did not improve from 0.64675
266/266 [=====] - 23s 87ms/step - loss: 0.6620 - precision: 0.8255 - recall: 0.8255
Epoch 24/30
262/266 [=====>.] - ETA: 0s - loss: 0.6667 - precision: 0.8214 - recall: 0.8214
Epoch 24: val_loss did not improve from 0.64675
266/266 [=====] - 24s 90ms/step - loss: 0.6651 - precision: 0.8218 - recall: 0.8218
Epoch 25/30
266/266 [=====] - ETA: 0s - loss: 0.6620 - precision: 0.8192 - recall: 0.8192
Epoch 25: val_loss did not improve from 0.64675
266/266 [=====] - 24s 90ms/step - loss: 0.6620 - precision: 0.8192 - recall: 0.8192
Epoch 26/30
264/266 [=====>.] - ETA: 0s - loss: 0.6630 - precision: 0.8221 - recall: 0.8221

```

Epoch 26: val_loss did not improve from 0.64675
266/266 [=====] - 14s 55ms/step - loss: 0.6636 - precision: 0.8220 - recall: 0.8220
Epoch 27/30
266/266 [=====] - ETA: 0s - loss: 0.6562 - precision: 0.8221 - recall: 0.8221
Epoch 27: val_loss did not improve from 0.64675
266/266 [=====] - 24s 91ms/step - loss: 0.6562 - precision: 0.8221 - recall: 0.8221
Epoch 28/30
264/266 [=====>.] - ETA: 0s - loss: 0.6545 - precision: 0.8258 - recall: 0.8258
Epoch 28: val_loss did not improve from 0.64675
266/266 [=====] - 14s 54ms/step - loss: 0.6543 - precision: 0.8259 - recall: 0.8259
Epoch 29/30
264/266 [=====>.] - ETA: 0s - loss: 0.6576 - precision: 0.8259 - recall: 0.8259
Epoch 29: val_loss did not improve from 0.64675
266/266 [=====] - 16s 61ms/step - loss: 0.6561 - precision: 0.8265 - recall: 0.8265
Epoch 30/30
266/266 [=====] - ETA: 0s - loss: 0.6533 - precision: 0.8234 - recall: 0.8234
Epoch 30: val_loss did not improve from 0.64675
266/266 [=====] - 15s 56ms/step - loss: 0.6533 - precision: 0.8234 - recall: 0.8234
*****
***** evaluating model... *****
*****
*****
Validation
1219/1219 [=====] - 7s 6ms/step - loss: 0.6585 - precision: 0.8038 - recall: 0.8038
loss: 0.6584734320640564
precision: 0.8037974834442139
recall: 0.7292863130569458
categorical_accuracy: 0.7735849022865295
one_hot_io_u: 0.5688682794570923

*****
***** saving parameters... *****
*****
***** saving model config and history object... *****
*****
***** saving plots... *****
Saving plots and model visualization at /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter 6
*****
***** saving models... *****
*****

```



1.5.6 Save the config file

```
drive_config_file = Path(dnn_config.MODEL_DIR / f"{str(config_file).split('/')[1]}")

# Create the target directory if it doesn't exist
drive_config_file.parent.mkdir(parents=True, exist_ok=True)

# Copy the file
shutil.copy(config_file, drive_config_file)

print(f"File copied from {config_file} to {drive_config_file}")
```

File copied from servir-aces/config.env to /content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output/dnn_v1/logs'

1.5.7 Load the logs files via TensorBoard

```
log_dir_dnn = f"{str(dnn_config.MODEL_DIR)}/logs"
log_dir_dnn
```

'/content/drive/MyDrive/Colab Notebooks/DL_Book/Chapter_1/output/dnn_v1/logs'

```
%tensorboard --logdir "{log_dir_dnn}"
```

<IPython.core.display.Javascript object>

1.5.8 Load the Saved DNN Model

```
dnn_model = tf.keras.models.load_model(f"{str(dnn_config.MODEL_DIR)}/trained-model")

print(dnn_model.summary())
```

Model: "model"

Layer (type)	Output Shape	Param #
=====		

input_layer (InputLayer)	[(None, None, 8)]	0
dense (Dense)	(None, None, 256)	2304
dropout (Dropout)	(None, None, 256)	0
dense_1 (Dense)	(None, None, 128)	32896
dropout_1 (Dropout)	(None, None, 128)	0
dense_2 (Dense)	(None, None, 64)	8256
dropout_2 (Dropout)	(None, None, 64)	0
dense_3 (Dense)	(None, None, 32)	2080
dropout_3 (Dropout)	(None, None, 32)	0
dense_4 (Dense)	(None, None, 5)	165

```

=====
Total params: 45701 (178.52 KB)
Trainable params: 45701 (178.52 KB)
Non-trainable params: 0 (0.00 Byte)
-----
None

```

1.5.9 Inference using Saved DNN Model

Now we can use the saved model to start the export of the prediction of the image. For prediction, you would need to first prepare your image data. We have already exported the image needed here, which we will use for now. See [this notebook](#) to understand how we did it.

In addition, [this notebook](#) shows how you can then use the image to predict from the saved Model.

In any case, you now have the prediction in the Earth Engine as image.

1.6 Independent Validation

For independent validation, we will use a file that we have prepared. These files were collected using [Collect Earth Online](#) by SCO and NASA DEVELOP interns. We will be using GEE here. Before we do that, let's make changes in our config file.

We will make sure our GCS_PROJECT is setup correctly.

```
GCS_PROJECT = "servir-ee"
```

1.6.1 Update the config file

```
GCS_PROJECT = "servir-ee" # @param {type:"string"}
```

```
config_settings = {  
    "GCS_PROJECT": GCS_PROJECT,  
}
```

```
for config_key in config_settings:  
    dotenv.set_key(dotenv_path=config_file,  
                   key_to_set=config_key,  
                   value_to_set=config_settings[config_key]  
                   )
```

1.6.2 Load config file variable

```
config = Config(config_file=config_file, override=True)
```

```
BASEDIR: /content
```

```
DATADIR: /content/datasets/dnn_planet_wo_indices
```

```
using features: ['red_before', 'green_before', 'blue_before', 'nir_before', 'red_during', 'g
```

```
using labels: ['class']
```

1.6.3 Import earthengine and geemap for visualization

```
# Import, authenticate and initialize the Earth Engine library.
import ee
ee.Authenticate()
EEUtils.initialize_session(use_highvolume=True, project=config.GCS_PROJECT)
```

```
import geemap
```

```
Map = geemap.Map()
```

1.6.4 Class Information and Masking

```
# CLASS
# 0 - cropland etc.
# 1 - rice
# 2 - forest
# 3 - Built up
# 4 - Others (includes water body)
l1 = ee.FeatureCollection("projects/servir-sco-assets/assets/Bhutan/BT_Admin_1")
paro = l1.filter(ee.Filter.eq("ADM1_EN", "Paro"))

# mask the rice growing zone
# in Paro, rice grows upto 2600 m asl (double check to make sure??)
dem = ee.Image("MERIT/DEM/v1_0_3") # ee.Image('USGS/SRTMGL1_003')
dem = dem.clip(paro)
rice_zone = dem.gte(0).And(dem.lte(2600))
```

<IPython.core.display.HTML object>

1.6.5 Model: U-Net

1.6.5.1 Load and visualize the prediction output

```
UNET_RGBN = ee.Image("projects/servir-ee/assets/dl-book/chapter-1/prediction/prediction_unet")
UNET_RGBN = UNET_RGBN.updateMask(rice_zone)
Map.centerObject(UNET_RGBN, 11)
Map.addLayer(UNET_RGBN.clip(paro), {"bands": ["prediction"], "min":0, "max":4, "palette": ["1", "2", "3", "4"]}, "Prediction")
Map
```

<IPython.core.display.HTML object>

```
Map(center=[27.378354616518475, 89.42005508391453], controls=(WidgetControl(options=['positioning'])
```

1.6.5.2 Calculate classification metrics

Remapping to rice and non-rice output

```
UNET_RGBN_remapped = UNET_RGBN.remap([0, 1, 2, 3, 4], [0, 1, 0, 0, 0], 0, "prediction")
Map.addLayer(UNET_RGBN_remapped, {"min": 0, "max": 1, "palette": ["cfcf00", "267300"]}, "UNET_RGBN_remapped")
Map
```

<IPython.core.display.HTML object>

```
Map(bottom=220961.0, center=[27.378354616518475, 89.42005508391453], controls=(WidgetControl(options=['positioning'])
```

```
sampling_geom = ee.FeatureCollection("projects/servir-ee/assets/dl-book/chapter-1/data/sampling_geom")
ceo_final_data = ee.FeatureCollection("projects/servir-ee/assets/dl-book/chapter-1/data/ceo_final_data")
ceo_final_data = ee.FeatureCollection(ceo_final_data.filter(ee.Filter.bounds(sampling_geom)))
```

<IPython.core.display.HTML object>

```
prediction_unet = UNET_RGBN_remapped.sampleRegions(
    collection = ceo_final_data,
    scale = 10,
    geometries = True
)

# print("predictionOutputUnet", prediction_unet.getInfo())
```

<IPython.core.display.HTML object>

```
error_matrix_unet = prediction_unet.errorMatrix(actual="rice", predicted="remapped")
test_acc_unet = error_matrix_unet.accuracy()
test_kappa_unet = error_matrix_unet.kappa()
test_recall_producer_acc_unet = error_matrix_unet.producersAccuracy().get([1, 0])
test_precision_consumer_acc_unet = error_matrix_unet.consumersAccuracy().get([0, 1])
f1_unet = error_matrix_unet.fscore().get([1])
```

<IPython.core.display.HTML object>

```
print("error_matrix_unet", error_matrix_unet.getInfo())
print("test_acc_unet", test_acc_unet.getInfo())
print("test_kappa_unet", test_kappa_unet.getInfo())
print("test_recall_producer_acc_unet", test_recall_producer_acc_unet.getInfo())
print("test_precision_consumer_acc_unet", test_precision_consumer_acc_unet.getInfo())
print("f1_unet", f1_unet.getInfo())
```

<IPython.core.display.HTML object>

```
error_matrix_unet [[1191, 29], [33, 50]]
test_acc_unet 0.9524174980813507
test_kappa_unet 0.5919321924312524
test_recall_producer_acc_unet 0.6024096385542169
test_precision_consumer_acc_unet 0.6329113924050633
f1_unet 0.6172839506172839
```

1.6.5.3 Calculate Probability Distribution

```
prob_output_unet = UNET_RGBN.select(["prediction", "others_etc", "cropland_etc", "urban", "f
                                .rename(["prediction_class", "others_prob", "cropland_prob", "ur
                                .sampleRegions(collection=ceo_final_data, scale=10, geometries=T

# print("prob_output_unet", prob_output_unet.getInfo())
```

<IPython.core.display.HTML object>

```
prob_output_unet = prob_output_unet.getInfo()
```

<IPython.core.display.HTML object>

1.6.6 Model: DNN

1.6.6.1 Load and visualize the prediction output

```
DNN_RGBN = ee.Image("projects/servir-ee/assets/dl-book/chapter-1/prediction/prediction_dnn_v
DNN_RGBN = DNN_RGBN.updateMask(rice_zone)
Map.centerObject(DNN_RGBN)
Map.addLayer(DNN_RGBN.clip(paro), {"bands": ["prediction"], "min":0, "max":4, "palette": ["F
Map
```

<IPython.core.display.HTML object>

```
Map(bottom=220961.0, center=[27.378354616518475, 89.42005508391453], controls=(WidgetControl
```

1.6.6.2 Calculate classification metrics

```
DNN_RGBN_remapped = DNN_RGBN.remap([0, 1, 2, 3, 4], [0, 1, 0, 0, 0], 0, "prediction")
Map.addLayer(DNN_RGBN_remapped, {"min": 0, "max": 1, "palette": ["cfcf00", "267300"]}, "DNN_I
Map
```

<IPython.core.display.HTML object>

```
Map(bottom=220961.0, center=[27.37845188654284, 89.42005507220328], controls=(WidgetControl(
```

```
prediction_dnn = DNN_RGBN_remapped.sampleRegions(
    collection = ceo_final_data,
    scale = 10,
    geometries = True
)

# print("predictionOutputDNN", prediction_dnn.getInfo())
```

<IPython.core.display.HTML object>

```
error_matrix_dnn = prediction_dnn.errorMatrix(actual="rice", predicted="remapped")
test_acc_dnn = error_matrix_dnn.accuracy()
test_kappa_dnn = error_matrix_dnn.kappa()
test_recall_producer_acc_dnn = error_matrix_dnn.producersAccuracy().get([1, 0])
test_precision_consumer_acc_dnn = error_matrix_dnn.consumersAccuracy().get([0, 1])
f1_dnn = error_matrix_dnn.fscore().get([1])
```

<IPython.core.display.HTML object>

```
print("error_matrix_dnn", error_matrix_dnn.getInfo())
print("test_acc_dnn", test_acc_dnn.getInfo())
print("test_kappa_dnn", test_kappa_dnn.getInfo())
print("test_recall_producer_acc_dnn", test_recall_producer_acc_dnn.getInfo())
print("test_precision_consumer_acc_dnn", test_precision_consumer_acc_dnn.getInfo())
print("f1_dnn", f1_dnn.getInfo())
```

<IPython.core.display.HTML object>

```
error_matrix_dnn [[1175, 45], [20, 63]]
test_acc_dnn 0.9501151189562548
test_kappa_dnn 0.6332676611314382
test_recall_producer_acc_dnn 0.7590361445783133
test_precision_consumer_acc_dnn 0.5833333333333334
f1_dnn 0.6596858638743456
```

1.6.6.3 Calculate Probability Distribution

```
prob_output_dnn = DNN_RGBN.select(["prediction", "others_etc", "cropland_etc", "urban", "forest_etc"]
                                   .rename(["prediction_class", "others_prob", "cropland_prob", "urban_prob", "forest_etc_prob"])
                                   .sampleRegions(collection=ceo_final_data, scale=10, geometries=True))

# print("prob_output_dnn", prob_output_dnn.getInfo())
```

<IPython.core.display.HTML object>

```
prob_output_dnn = prob_output_dnn.getInfo()
```

<IPython.core.display.HTML object>

1.7 Figures and Plots

1.7.1 Training and Validation Plot

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pickle
```

```
%matplotlib inline
```

<IPython.core.display.HTML object>

```
with open(unet_config.MODEL_DIR / "model.pkl", "rb") as f:
    unet_model_metrics = pickle.load(f)

with open(dnn_config.MODEL_DIR / "model.pkl", "rb") as f:
    dnn_model_metrics = pickle.load(f)
```

<IPython.core.display.HTML object>

```
# Create subplots for different metrics in a 3x4 grid
fig, axs = plt.subplots(2, 4, figsize=(4*7, 6*2))

colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
metrics = ["loss", "precision", "recall", "categorical_accuracy"]
metrics_name = ["Loss", "Precision", "Recall", "Categorical Accuracy"]

epochs = range(1, config.EPOCHS + 1)

title_fontsize = 22
label_fontsize = 22
legend_fontsize = 15
tick_fontsize = 18
lw=1.5

for i in range(2):
    for y in range(len(metrics)):
        if i == 1:
            axs[i][y].plot(epochs, unet_model_metrics[f"val_{metrics[y]}"], color=colors[0],
                           axs[i][y].plot(epochs, dnn_model_metrics[f"val_{metrics[y]}"], color=colors[1],
            axs[i][y].set_title(f"Validate {metrics_name[y]}", fontsize=title_fontsize)
            axs[i][y].set_xlabel("epochs", fontsize=label_fontsize)
            axs[i][y].set_ylabel(f"{metrics[y]}", fontsize=label_fontsize)
```



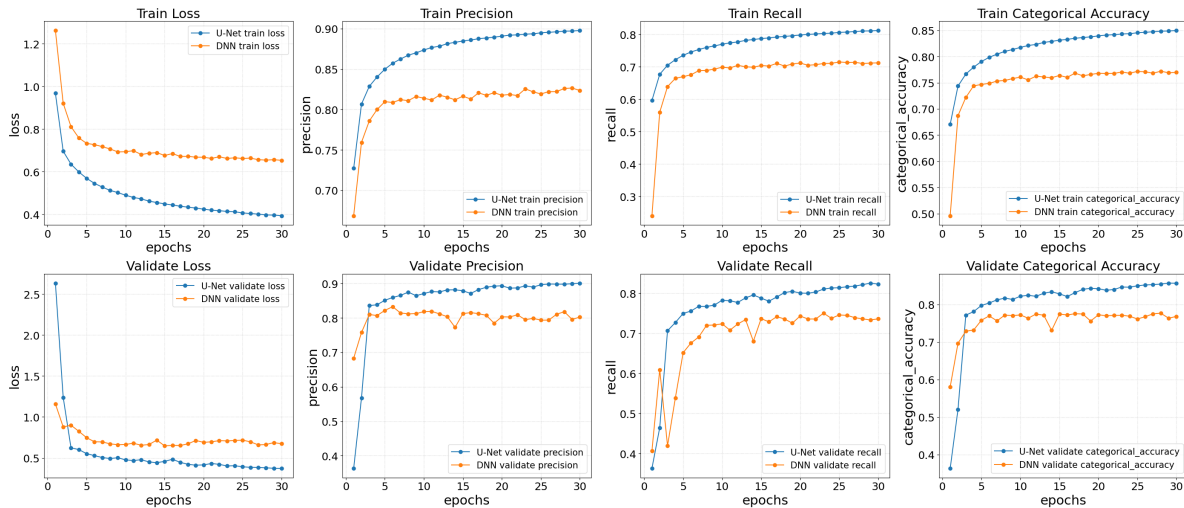
```

        axs[i][y].grid(linestyle="dotted", alpha=0.7)
        axs[i][y].legend(fontsize=legend_fontsize)
        axs[i][y].tick_params(axis="both", which="major", labelsize=tick_fontsize)
    else:
        axs[i][y].plot(epochs, unet_model_metrics[metrics[y]], color=colors[0], lw=lw, ma=ma)
        axs[i][y].plot(epochs, dnn_model_metrics[metrics[y]], color=colors[1], lw=lw, ma=ma)
        axs[i][y].set_title(f"Train {metrics_name[y]}", fontsize=title_fontsize)
        axs[i][y].set_xlabel("epochs", fontsize=label_fontsize)
        axs[i][y].set_ylabel(f"{metrics[y]}", fontsize=label_fontsize)
        axs[i][y].grid(linestyle="dotted", alpha=0.7)
        axs[i][y].legend(fontsize=legend_fontsize)
        axs[i][y].tick_params(axis="both", which="major", labelsize=tick_fontsize)

# Adjust layout and show the plot
plt.tight_layout()
# plt.savefig("metrics_plot_model_comparison.png", dpi=500, bbox_inches="tight")
plt.show()

```

<IPython.core.display.HTML object>



```

# Create subplots for different metrics in a 3x4 grid
fig, axs = plt.subplots(1, 4, figsize=(4*7, 6*1))

colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
metrics = ["loss", "precision", "recall", "categorical_accuracy"]

```

```

metrics_name = ["Loss", "Precision", "Recall", "Categorical Accuracy"]

epochs = range(1, config.EPOCHS + 1)

title_fontsize = 22
label_fontsize = 22
legend_fontsize = 15
tick_fontsize = 18
lw=1.5

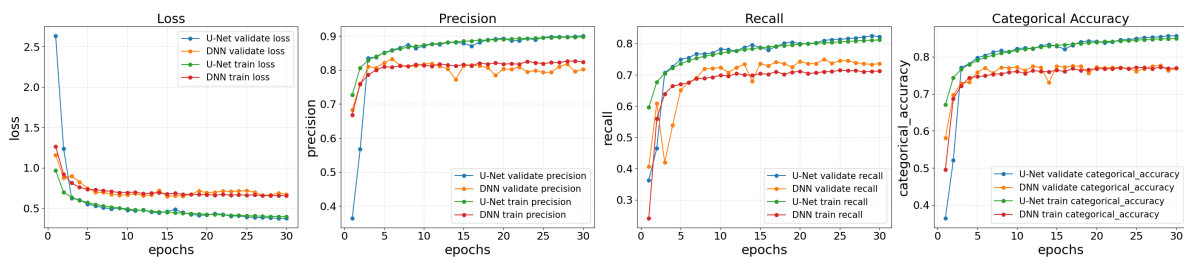
for y in range(len(metrics)):
    axs[y].plot(epochs, unet_model_metrics[f"val_{metrics[y]}"], color=colors[0], marker="o", lw=lw)
    axs[y].plot(epochs, dnn_model_metrics[f"val_{metrics[y]}"], color=colors[1], lw=lw, marker="o", lw=lw)

    axs[y].plot(epochs, unet_model_metrics[metrics[y]], color=colors[2], lw=lw, marker="o", lw=lw)
    axs[y].plot(epochs, dnn_model_metrics[metrics[y]], color=colors[3], lw=lw, marker="o", lw=lw)
    axs[y].set_title(f"{metrics_name[y]}", fontsize=title_fontsize)
    axs[y].set_xlabel("epochs", fontsize=label_fontsize)
    axs[y].set_ylabel(f"{metrics[y]}", fontsize=label_fontsize)
    axs[y].grid(linestyle="dotted", alpha=0.7)
    axs[y].legend(fontsize=legend_fontsize)
    axs[y].tick_params(axis="both", which="major", labelsize=tick_fontsize)

# Adjust layout and show the plot
plt.tight_layout()
# plt.savefig("metrics_plot_model_comparison.png", dpi=500, bbox_inches="tight")
plt.show()

```

<IPython.core.display.HTML object>



1.7.2 Probability Distribution Plot

```
all_data = {}

UNET_DATA = []
DNN_DATA = []

UNET_RICE_DATA = []
DNN_RICE_DATA = []

UNET_OTHER_DATA = []
DNN_OTHER_DATA = []

for i, feature in enumerate(prob_output_unet["features"]):
    UNET_RICE_PROB = round(feature["properties"]["rice_prob"], 5)
    UNET_OTHER_PROB = round(feature["properties"]["cropland_prob"] + round(feature["properties"], 5))
    UNET_DATA.append([UNET_RICE_PROB, UNET_OTHER_PROB])
    UNET_RICE_DATA.append(UNET_RICE_PROB)
    UNET_OTHER_DATA.append(UNET_OTHER_PROB)

    DNN_FEATURE = prob_output_dnn["features"][i]
    DNN_RICE_PROB = round(DNN_FEATURE["properties"]["rice_prob"], 5)
    DNN_OTHER_PROB = 1. - round(DNN_FEATURE["properties"]["rice_prob"], 5)
    # DNN_OTHER_PROB = round(DNN_FEATURE["properties"]["cropland_prob"] + DNN_FEATURE["properties"], 5)
    DNN_DATA.append([DNN_RICE_PROB, DNN_OTHER_PROB])
    DNN_RICE_DATA.append(DNN_RICE_PROB)
    DNN_OTHER_DATA.append(DNN_OTHER_PROB)
```

<IPython.core.display.HTML object>

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(8, 5))

title_fontsize = 22
label_fontsize = 10
tick_fontsize = 10

# rectangular box plot
bplot1 = ax1.boxplot([UNET_RICE_DATA, DNN_RICE_DATA],
                    notch=True,
                    vert=True, # vertical box alignment
                    patch_artist=True, # fill with color
```

```

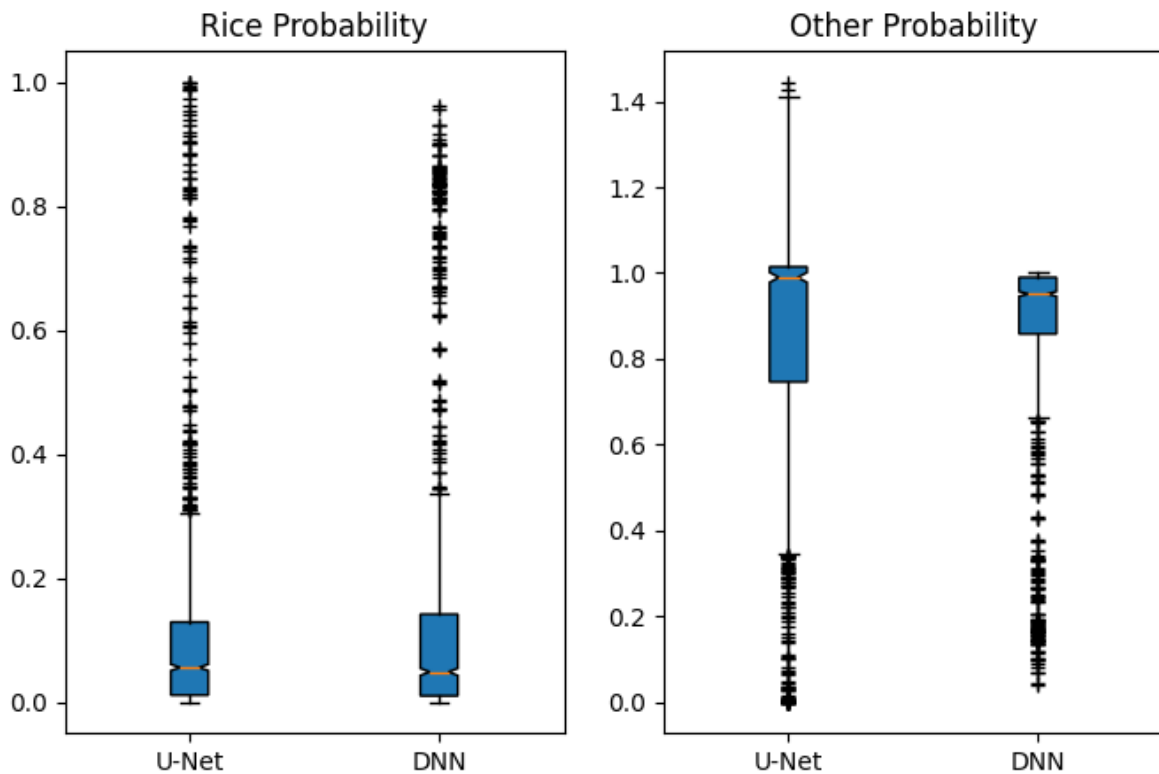
        labels=["U-Net", "DNN"],
        sym="k+") # will be used to label x-ticks
ax1.set_title("Rice Probability")

# notch shape box plot
bplot2 = ax2.boxplot([unet_other_data, dnn_other_data],
                    notch=True, # notch shape
                    vert=True, # vertical box alignment
                    patch_artist=True, # fill with color
                    labels=["U-Net", "DNN"],
                    sym="k+") # will be used to label x-ticks
ax2.set_title("Other Probability")

```

<IPython.core.display.HTML object>

Text(0.5, 1.0, 'Other Probability')



2 Copyright 2026 - Osmar Yupanqui & Marvin Quispe

```
# Conservación Amazónica - ACCA
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

3 Selective Logging Detection with Deep Learning and Very-High Resolution Imagery

3.0.0.1 Authors: Osmar Yupanqui and Marvin Quispe

3.1 Part 2: Modeling

3.1.1 This notebook shows the workflow used to train a deep learning model with the patches saved from the previous notebook (Notebook n.^º1). We will load our TFRecords, train a model with them and save it for further use (inference). The notebook was developed and configured specifically to run on Google Colab, so its implementation is optimized to run on that platform. In addition, it is necessary to configure the environment to enable GPU computing by changing the runtime type. First, navigate to *Edit* → *Notebook Settings* and select GPU from the Hardware Accelerator options.

3.2 1. Data collection and initial settings

We are going to connect **Google Drive** with **Google Colab** to manage the exported data (patches) from the previous Notebook.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Let's see the contents of our Drive folder using the linux command `!ls`. Note: This command only runs on Google Colab

```
!ls -F /content/drive/MyDrive/DL_Book
```

img/	train_14.tfrecord.gz	train_6.tfrecord.gz
notebooks/	train_150.tfrecord.gz	train_70.tfrecord.gz
pictures/	train_151.tfrecord.gz	train_71.tfrecord.gz
testing_0.tfrecord.gz	train_152.tfrecord.gz	train_72.tfrecord.gz
testing_10.tfrecord.gz	train_153.tfrecord.gz	train_73.tfrecord.gz
testing_11.tfrecord.gz	train_154.tfrecord.gz	train_74.tfrecord.gz
testing_12.tfrecord.gz	train_155.tfrecord.gz	train_75.tfrecord.gz
testing_13.tfrecord.gz	train_156.tfrecord.gz	train_76.tfrecord.gz
testing_14.tfrecord.gz	train_157.tfrecord.gz	train_77.tfrecord.gz
testing_15.tfrecord.gz	train_158.tfrecord.gz	train_78.tfrecord.gz
testing_16.tfrecord.gz	train_159.tfrecord.gz	train_79.tfrecord.gz
testing_17.tfrecord.gz	train_15.tfrecord.gz	train_7.tfrecord.gz
testing_18.tfrecord.gz	train_160.tfrecord.gz	train_80.tfrecord.gz
testing_19.tfrecord.gz	train_161.tfrecord.gz	train_81.tfrecord.gz
testing_1.tfrecord.gz	train_162.tfrecord.gz	train_82.tfrecord.gz
testing_20.tfrecord.gz	train_163.tfrecord.gz	train_83.tfrecord.gz
testing_21.tfrecord.gz	train_164.tfrecord.gz	train_84.tfrecord.gz
testing_22.tfrecord.gz	train_165.tfrecord.gz	train_85.tfrecord.gz
testing_23.tfrecord.gz	train_166.tfrecord.gz	train_86.tfrecord.gz
testing_24.tfrecord.gz	train_167.tfrecord.gz	train_87.tfrecord.gz
testing_2.tfrecord.gz	train_168.tfrecord.gz	train_88.tfrecord.gz
testing_3.tfrecord.gz	train_169.tfrecord.gz	train_89.tfrecord.gz
testing_4.tfrecord.gz	train_16.tfrecord.gz	train_8.tfrecord.gz
testing_5.tfrecord.gz	train_170.tfrecord.gz	train_90.tfrecord.gz
testing_6.tfrecord.gz	train_171.tfrecord.gz	train_91.tfrecord.gz
testing_7.tfrecord.gz	train_17.tfrecord.gz	train_92.tfrecord.gz
testing_8.tfrecord.gz	train_18.tfrecord.gz	train_93.tfrecord.gz
testing_9.tfrecord.gz	train_19.tfrecord.gz	train_94.tfrecord.gz
train_0.tfrecord.gz	train_1.tfrecord.gz	train_95.tfrecord.gz
train_100.tfrecord.gz	train_20.tfrecord.gz	train_96.tfrecord.gz
train_101.tfrecord.gz	train_21.tfrecord.gz	train_97.tfrecord.gz
train_102.tfrecord.gz	train_22.tfrecord.gz	train_98.tfrecord.gz
train_103.tfrecord.gz	train_23.tfrecord.gz	train_99.tfrecord.gz
train_104.tfrecord.gz	train_24.tfrecord.gz	train_9.tfrecord.gz
train_105.tfrecord.gz	train_25.tfrecord.gz	validation_0.tfrecord.gz
train_106.tfrecord.gz	train_26.tfrecord.gz	validation_10.tfrecord.gz
train_107.tfrecord.gz	train_27.tfrecord.gz	validation_11.tfrecord.gz
train_108.tfrecord.gz	train_28.tfrecord.gz	validation_12.tfrecord.gz
train_109.tfrecord.gz	train_29.tfrecord.gz	validation_13.tfrecord.gz
train_10.tfrecord.gz	train_2.tfrecord.gz	validation_14.tfrecord.gz
train_110.tfrecord.gz	train_30.tfrecord.gz	validation_15.tfrecord.gz
train_111.tfrecord.gz	train_31.tfrecord.gz	validation_16.tfrecord.gz
train_112.tfrecord.gz	train_32.tfrecord.gz	validation_17.tfrecord.gz

train_113.tfrecord.gz	train_33.tfrecord.gz	validation_18.tfrecord.gz
train_114.tfrecord.gz	train_34.tfrecord.gz	validation_19.tfrecord.gz
train_115.tfrecord.gz	train_35.tfrecord.gz	validation_1.tfrecord.gz
train_116.tfrecord.gz	train_36.tfrecord.gz	validation_20.tfrecord.gz
train_117.tfrecord.gz	train_37.tfrecord.gz	validation_21.tfrecord.gz
train_118.tfrecord.gz	train_38.tfrecord.gz	validation_22.tfrecord.gz
train_119.tfrecord.gz	train_39.tfrecord.gz	validation_23.tfrecord.gz
train_11.tfrecord.gz	train_3.tfrecord.gz	validation_24.tfrecord.gz
train_120.tfrecord.gz	train_40.tfrecord.gz	validation_25.tfrecord.gz
train_121.tfrecord.gz	train_41.tfrecord.gz	validation_26.tfrecord.gz
train_122.tfrecord.gz	train_42.tfrecord.gz	validation_27.tfrecord.gz
train_123.tfrecord.gz	train_43.tfrecord.gz	validation_28.tfrecord.gz
train_124.tfrecord.gz	train_44.tfrecord.gz	validation_29.tfrecord.gz
train_125.tfrecord.gz	train_45.tfrecord.gz	validation_2.tfrecord.gz
train_126.tfrecord.gz	train_46.tfrecord.gz	validation_30.tfrecord.gz
train_127.tfrecord.gz	train_47.tfrecord.gz	validation_31.tfrecord.gz
train_128.tfrecord.gz	train_48.tfrecord.gz	validation_32.tfrecord.gz
train_129.tfrecord.gz	train_49.tfrecord.gz	validation_33.tfrecord.gz
train_12.tfrecord.gz	train_4.tfrecord.gz	validation_34.tfrecord.gz
train_130.tfrecord.gz	train_50.tfrecord.gz	validation_35.tfrecord.gz
train_131.tfrecord.gz	train_51.tfrecord.gz	validation_36.tfrecord.gz
train_132.tfrecord.gz	train_52.tfrecord.gz	validation_37.tfrecord.gz
train_133.tfrecord.gz	train_53.tfrecord.gz	validation_38.tfrecord.gz
train_134.tfrecord.gz	train_54.tfrecord.gz	validation_39.tfrecord.gz
train_135.tfrecord.gz	train_55.tfrecord.gz	validation_3.tfrecord.gz
train_136.tfrecord.gz	train_56.tfrecord.gz	validation_40.tfrecord.gz
train_137.tfrecord.gz	train_57.tfrecord.gz	validation_41.tfrecord.gz
train_138.tfrecord.gz	train_58.tfrecord.gz	validation_42.tfrecord.gz
train_139.tfrecord.gz	train_59.tfrecord.gz	validation_43.tfrecord.gz
train_13.tfrecord.gz	train_5.tfrecord.gz	validation_44.tfrecord.gz
train_140.tfrecord.gz	train_60.tfrecord.gz	validation_45.tfrecord.gz
train_141.tfrecord.gz	train_61.tfrecord.gz	validation_46.tfrecord.gz
train_142.tfrecord.gz	train_62.tfrecord.gz	validation_47.tfrecord.gz
train_143.tfrecord.gz	train_63.tfrecord.gz	validation_48.tfrecord.gz
train_144.tfrecord.gz	train_64.tfrecord.gz	validation_4.tfrecord.gz
train_145.tfrecord.gz	train_65.tfrecord.gz	validation_5.tfrecord.gz
train_146.tfrecord.gz	train_66.tfrecord.gz	validation_6.tfrecord.gz
train_147.tfrecord.gz	train_67.tfrecord.gz	validation_7.tfrecord.gz
train_148.tfrecord.gz	train_68.tfrecord.gz	validation_8.tfrecord.gz
train_149.tfrecord.gz	train_69.tfrecord.gz	validation_9.tfrecord.gz

Our folder should contain several files whose names begin with: - train - testing - validation

Those files are our exported patches from the previous Notebook. If you do not wish to run the previous notebook, you can create a copy from the original folder. Click [here](#) to go to the Drive folder. Make sure to modify the paths to match the current Notebook.

We will also import some libraries that will contribute to the training of the model. This notebook was built and tested using Google Colab, so the libraries used correspond to the latest version available on this platform at the time of construction and testing (e.g. Tensorflow 2.19.0). Check the latest versions of TensorFlow [here](#).

```
import tensorflow as tf
print(tf.__version__)
```

2.19.0

[Numpy](#) is another important library, that allow us to perform operations with tensors, matrices and vectors.

```
import numpy as np
```

We will also import [Matplotlib](#), for the data visualization.

```
import matplotlib.pyplot as plt
```

Finally, we will also import [os](#), to read files and manipulate paths.

```
import os
```

3.3 2. Data visualization

We will start by viewing a single TFRecord file, if we explore the folder where the files were downloaded, we can see that they are “compressed” with a special `.gz` format, Tensorflow allows the use of this special format, without the need to decompress the files.

We will start by creating a variable called `record`, which will include in a container `tf.data.TFRecordDataset()` the path of an exported file. *Please verify that the export path is the same, in the previous example we exported our data to the `DL_Book` folder in Google Drive*

```
record = tf.data.TFRecordDataset('/content/drive/MyDrive/DL_Book/testing_0.tfrecord.gz', comp
print(record)
```

```
<TFRecordDatasetV2 element_spec=TensorSpec(shape=(), dtype=tf.string, name=None)>
```

We can see that the `TFRecordDataset` does not show all the information, we cannot know the **shape** of the tensors, even though we know based on the previous code that they have a 128 by 128 shape. We also cannot know the **dtype** (the data type) neither the number of elements that our `TFRecordDataset` has.

Because `TFRecords` are binary storage files, we must transform them into a readable structure. To do this, we will create a dictionary that contains the bands that we exported in the previous code and assign each band a shape with `tf.io.FixedLenFeature()` and a data type.

```
features = {
    'b1': tf.io.FixedLenFeature([128, 128], tf.float32),
    'b2': tf.io.FixedLenFeature([128, 128], tf.float32),
    'b3': tf.io.FixedLenFeature([128, 128], tf.float32),
    'b4': tf.io.FixedLenFeature([128, 128], tf.float32),
    'class': tf.io.FixedLenFeature([128, 128], tf.float32),
}
```

We will create a function to read a serialized example into the structure defined by the dictionary created above.

```
def parse_tfrecord(example_proto):
    return tf.io.parse_single_example(example_proto, features) # This parses each TFRecord to
```

We will apply the newly created function to our `TFRecordDataset` using the `.map()` function, which allows iterating through each element of our `TFRecordDataset`.

```
serialized = record.map(parse_tfrecord) # With .map() we loop through each element of our TF
print(serialized)
```

```
<_MapDataset element_spec={'b1': TensorSpec(shape=(128, 128), dtype=tf.float32, name=None),
```

We can see that the newly created object named `serialized` has a different form than the original `TFRecordDataset`. This new object has the structure defined by the dictionary created. To display the numerical values of each of the bands of our new object, we will use the function `get_single_element()`.

```
element = serialized.get_single_element() # With .get_single_element() we retrieve an indivi
print(element)
```

```

{'b1': <tf.Tensor: shape=(128, 128), dtype=float32, numpy=
array([[6311., 6311., 6311., ..., 6482., 6610., 6610.],
      [6354., 6269., 6269., ..., 6439., 6567., 6652.],
      [6311., 6226., 6141., ..., 6354., 6439., 6524.],
      ...,
      [5970., 5970., 6055., ..., 6183., 6183., 6141.],
      [6098., 6055., 6141., ..., 5970., 6098., 6226.],
      [6311., 6226., 6226., ..., 5885., 6055., 6226.]], dtype=float32)>, 'b2': <tf.Tensor: s
array([[5108., 5047., 4956., ..., 5169., 5260., 5351.],
      [5108., 5047., 4956., ..., 5169., 5199., 5290.],
      [5078., 5017., 4956., ..., 5169., 5199., 5290.],
      ...,
      [5169., 5199., 5290., ..., 5473., 5138., 4925.],
      [5290., 5290., 5321., ..., 5169., 4986., 4925.],
      [5473., 5442., 5412., ..., 5078., 4986., 4986.]], dtype=float32)>, 'b3': <tf.Tensor: s
array([[2867., 2748., 2628., ..., 2987., 3130., 3178.],
      [2843., 2724., 2580., ..., 2843., 2891., 2987.],
      [2748., 2652., 2580., ..., 2843., 2915., 2963.],
      ...,
      [2700., 2700., 2772., ..., 2748., 2557., 2461.],
      [2748., 2724., 2772., ..., 2652., 2580., 2557.],
      [2843., 2748., 2772., ..., 2604., 2604., 2604.]], dtype=float32)>, 'b4': <tf.Tensor: s
array([[12274., 11812., 11376., ..., 12172., 12403., 12737.],
      [12505., 11966., 11324., ..., 12659., 12968., 13250.],
      [12428., 11786., 11016., ..., 13019., 13327., 13533.],
      ...,
      [12172., 12505., 12968., ..., 14740., 13558., 12377.],
      [12839., 12865., 12942., ..., 13866., 12993., 12223.],
      [13610., 13301., 13045., ..., 12814., 12274., 12043.]]],
      dtype=float32)>, 'class': <tf.Tensor: shape=(128, 128), dtype=float32, numpy=
array([[0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      ...,
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.],
      [0., 0., 0., ..., 0., 0., 0.]], dtype=float32)>}]

```

The result is a dictionary, from which we can extract each band using `get()` followed by the key, which in this case is the name of the exported band.

```

b1 = element.get('b1')
b2 = element.get('b2')
b3 = element.get('b3')
b4 = element.get('b4')
label = element.get('class')

```

We can now view the numerical values of any exported band. We can also transform the objects to a numpy array using the `numpy()` function.

Now we will create an “image” to be able to visualize it, concatenating the values of the created bands, into a single tensor.

```

img = tf.constant([b3.numpy(), b2.numpy(), b1.numpy()]) # This appends each band to a constant
print(img)

```

```

tf.Tensor(
[[[2867. 2748. 2628. ... 2987. 3130. 3178.]
  [2843. 2724. 2580. ... 2843. 2891. 2987.]
  [2748. 2652. 2580. ... 2843. 2915. 2963.]
  ...
  [2700. 2700. 2772. ... 2748. 2557. 2461.]
  [2748. 2724. 2772. ... 2652. 2580. 2557.]
  [2843. 2748. 2772. ... 2604. 2604. 2604.]]

[[[5108. 5047. 4956. ... 5169. 5260. 5351.]
  [5108. 5047. 4956. ... 5169. 5199. 5290.]
  [5078. 5017. 4956. ... 5169. 5199. 5290.]
  ...
  [5169. 5199. 5290. ... 5473. 5138. 4925.]
  [5290. 5290. 5321. ... 5169. 4986. 4925.]
  [5473. 5442. 5412. ... 5078. 4986. 4986.]]

[[[6311. 6311. 6311. ... 6482. 6610. 6610.]
  [6354. 6269. 6269. ... 6439. 6567. 6652.]
  [6311. 6226. 6141. ... 6354. 6439. 6524.]
  ...
  [5970. 5970. 6055. ... 6183. 6183. 6141.]
  [6098. 6055. 6141. ... 5970. 6098. 6226.]
  [6311. 6226. 6226. ... 5885. 6055. 6226.]]], shape=(3, 128, 128), dtype=float32)

```

If we visualize the shape of the tensor, we see that it has the shape of (3, 128, 128), in order to plot it we must transform the shape to (128, 128, 3). To do this we will use the

`.transpose()` function. In addition, since Matplotlib does only tolerate values from 0 to 255, we will normalize our image (tensor) with `.divide()`.

```
imgRGB = tf.transpose(tf.divide(img, 12000), [1, 2, 0])
```

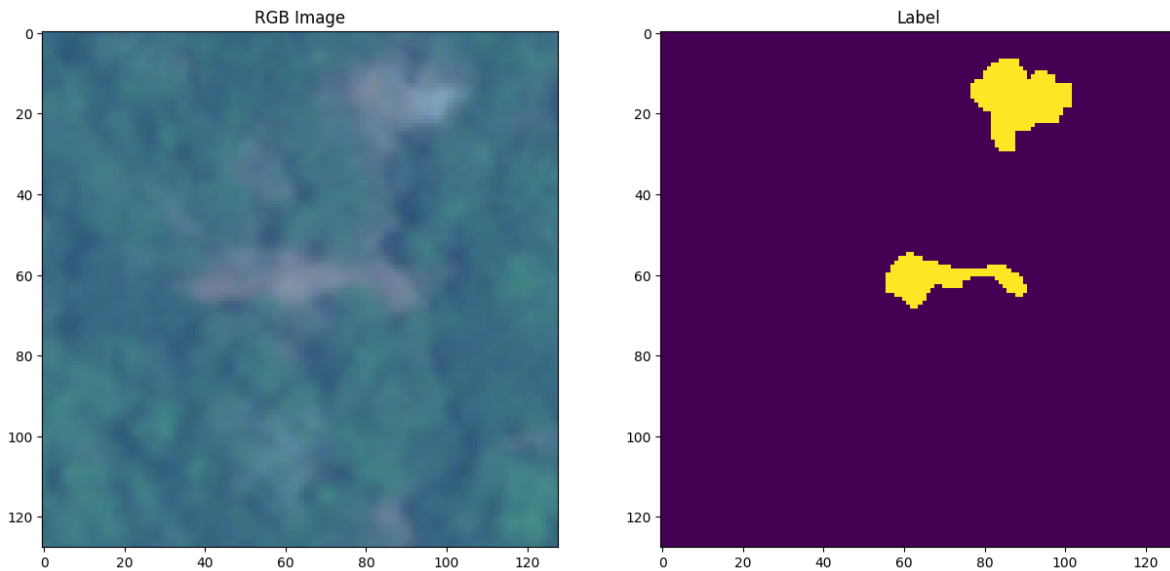
Now we will plot the results using the matplotlib library. We will be able to contrast how the logging activity is visualized with a SkySat (0.5 m) image (on the left), and the hand-digitized label (on the right).

```
fig, axs = plt.subplots(1, 2, figsize = (15, 15))

axs[0].set_title('RGB Image')
axs[0].imshow(imgRGB)

axs[1].set_title('Label')
axs[1].imshow(label.numpy())

plt.show()
```



This plot compares the exported SkySat satellite image patch with the hand-digitized label. Performing the manual digitization process takes many hours, especially when the logging activity is intense. Deep Learning models greatly speed up this manual process and allow the user to focus on evaluating the output.

3.4 3. Data processing

3.4.1 3.1 Definition of variables

Before performing data processing, some variables will be defined for the training. - The variable `kernel_size` contains the size of the patch, from which we build its `shape`. - The variable `patch_path` refers to the folder containing the previously exported patches. - `buffer_size` this number is the amount of data that will be filled when shuffling. For example, if our data contains 100 records and we have a `buffer_size` of 10, then `shuffle` will select a random element from the first 10 records. The space of this element will be replaced by the 101-st element. Since our dataset contains less than 250 files, and we want the model to shuffle randomly the **whole dataset** then we will define this parameter to 1000 (it could be higher).

The model also needs hyperparameters, defining these parameters can sometimes be time consuming and experimental. Some of the most important are described below: - `batch_size`: The number of examples used in a pass through the network. A high batch size can lead to faster training times but may result in lower accuracy and overfitting, while a small batch sizes can provide better accuracy, but can be computationally expensive and time-consuming. We will set our batch size to 4 (because this is a small dataset). - `epochs`: This is the amount of training cycles through all of the samples in the training dataset. This is how many times the model will see the entire training data before completing training. We will set this value to 10. - `learning_rate`: This controls how much the model's parameters are adjusted during each training step. This will be set to 0.1

```
bands = ['b1', 'b2', 'b3', 'b4']
response = ['class']
features = bands + response

kernel_size = 128
kernel_shape = [kernel_size, kernel_size]

columns = [
    tf.io.FixedLenFeature(shape = kernel_shape, dtype = tf.float32) for k in features
]

# Path to the folder with patches (Benchmark dataset)
patch_path = '/content/drive/MyDrive/DL_Book'
train_path = f'{patch_path}/train*'
val_path = f'{patch_path}/validation*'
test_path = f'{patch_path}/testing*'

features_dict = dict(zip(features, columns))
```

```

train_size = len([f for f in os.listdir(patch_path) if f.startswith('train')])
val_size = len([f for f in os.listdir(patch_path) if f.startswith('validation')])
test_size = len([f for f in os.listdir(patch_path) if f.startswith('testing')])

# Hyperparameters
batch_size = 4
epochs = 10
buffer_size = 1000
learning_rate = 0.1

```

3.4.2 3.2 Create a TFRecordDataset

TFRecordDataset are a collection of TFRecords, so they are very useful for storing large amounts of data, and are optimized to save memory.

First we will create a function called `.parse_tfrecord()`, which will perform the same functionality as in the data visualization, done previously. We will transform the structure of each TFRecord.

```

def parse_tfrecord(example_proto):
    return tf.io.parse_single_example(example_proto, features_dict)

```

Second, a function called `.to_tuple()` will be created, which will convert the tensor dictionary to a tuple, with the scheme (inputs, outputs).

```

def to_tuple(inputs):
    inputsList = [inputs.get(key) for key in features]
    stacked = tf.stack(inputsList, axis = 0) # This stacks the input list to a single tensor
    stacked = tf.transpose(stacked, [1, 2, 0]) # Transposition of the stacked tensor to the
    return stacked[:, :, :len(bands)], stacked[:, :, len(bands):]

```

Third, a function will be created to read the tensors, this function will also apply the functions created previously: `.parse_tfrecord()` and `.to_tuple()`

```

def get_dataset(pattern):
    glob = tf.io.gfile.glob(pattern) # This reads the files in our path
    dataset = tf.data.TFRecordDataset(glob, compression_type = 'GZIP') # Add the files to a
    dataset = dataset.map(parse_tfrecord, num_parallel_calls = 5)
    dataset = dataset.map(to_tuple, num_parallel_calls = 5)
    return dataset

```

We will create 3 TFRecordDataset, which will contain our training, validation and test data set, respectively. To do this we will use functions, within each one there will be an object called `glob` that will indicate the path of the data sets.

```
def get_patch_from_path(path, batch_size, buffer_size = 1000, shuffle = False, repeat = False):
    dataset = get_dataset(path + '*')
    dataset = dataset.shuffle(buffer_size, seed = 42) if shuffle else dataset # We will shuffle
    dataset = dataset.batch(batch_size)
    dataset = dataset.repeat() if repeat else dataset
    return dataset

training_data = get_patch_from_path(train_path, batch_size = batch_size, buffer_size = buffer_size)
validation_data = get_patch_from_path(val_path, batch_size = 1, repeat = True)
testing_data = get_patch_from_path(test_path, batch_size = 1)
```

If we print the 3 sets of data we will see that the training and validation data we see are of type `RepeatDataset()`, that is, they are files that are constantly being generated, which based on the previous functions are being ordered randomly and the Outgoing data set has a size of 1 (based on batch size).

```
print(training_data)
print(validation_data)
print(testing_data)
```

```
<_RepeatDataset element_spec=(TensorSpec(shape=(None, 128, 128, 4), dtype=tf.float32, name=None))
<_RepeatDataset element_spec=(TensorSpec(shape=(None, 128, 128, 4), dtype=tf.float32, name=None))
<_BatchDataset element_spec=(TensorSpec(shape=(None, 128, 128, 4), dtype=tf.float32, name=None))
```

3.5 4. Deep Learning

3.5.1 4.1 Model construction

In this notebook we will use a modification of the U-Net architecture developed by [Ronneberger et al., \(2015\)](#). The U-Net architecture was initially designed for use in biomedical imaging but proved to be very efficient in the segmentation of satellite and drone images, and is currently one of the most widely used for that purpose. This architecture has demonstrated remarkable effectiveness in various environmental monitoring applications, including agricultural field boundary detection [John & Zhang, 2022](#), urban land use classification, and coastal change detection. [Ulmas & Liiv, 2020](#) further validated U-Net's superiority in crop type mapping from multi-spectral satellite imagery, showing that its ability to preserve spatial